

INTRODUCTION TO BIOINFORMATICS



Rohan Mehta
Elmhurst University

Introduction to Bioinformatics

Rohan Mehta

Elmhurst University

This text is disseminated via the Open Education Resource (OER) LibreTexts Project (<https://LibreTexts.org>) and like the thousands of other texts available within this powerful platform, it is freely available for reading, printing, and "consuming."

The LibreTexts mission is to bring together students, faculty, and scholars in a collaborative effort to provide an accessible, and comprehensive platform that empowers our community to develop, curate, adapt, and adopt openly licensed resources and technologies; through these efforts we can reduce the financial burden born from traditional educational resource costs, ensuring education is more accessible for students and communities worldwide.

Most, but not all, pages in the library have licenses that may allow individuals to make changes, save, and print this book. Carefully consult the applicable license(s) before pursuing such effects. Instructors can adopt existing LibreTexts texts or Remix them to quickly build course-specific resources to meet the needs of their students. Unlike traditional textbooks, LibreTexts' web based origins allow powerful integration of advanced features and new technologies to support learning.



LibreTexts is the adaptable, user-friendly non-profit open education resource platform that educators trust for creating, customizing, and sharing accessible, interactive textbooks, adaptive homework, and ancillary materials. We collaborate with individuals and organizations to champion open education initiatives, support institutional publishing programs, drive curriculum development projects, and more.

The LibreTexts libraries are Powered by [NICE CXone Expert](#) and was supported by the Department of Education Open Textbook Pilot Project, the California Education Learning Lab, the UC Davis Office of the Provost, the UC Davis Library, the California State University Affordable Learning Solutions Program, and Merlot. This material is based upon work supported by the National Science Foundation under Grant No. 1246120, 1525057, and 1413739.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation nor the US Department of Education.

Have questions or comments? For information about adoptions or adaptations contact info@LibreTexts.org or visit our main website at <https://LibreTexts.org>.

This text was compiled on 07/03/2026

TABLE OF CONTENTS

[Licensing](#)

[1: Modules](#)

- [1.1: BLAST \(AKA so you have a sequence\)](#)
- [1.2: DNA Sequencing](#)
- [1.3: Genome Assembly](#)
- [1.4: Genome Alignment](#)
- [1.5: Gene Annotation](#)
- [1.6: Gene Expression Analysis](#)
- [1.7: Protein Structure](#)

[Index](#)

[Glossary](#)

[Detailed Licensing](#)

[Detailed Licensing](#)

Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

CHAPTER OVERVIEW

1: Modules

- [1.1: BLAST \(AKA so you have a sequence\)](#)
- [1.2: DNA Sequencing](#)
- [1.3: Genome Assembly](#)
- [1.4: Genome Alignment](#)
- [1.5: Gene Annotation](#)
- [1.6: Gene Expression Analysis](#)
- [1.7: Protein Structure](#)

[1: Modules](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by LibreTexts.

? Exercise 1.1.1

Using the above matrices, compute the match scores for the following pairs of sequences:

- 1) ATCTACGG vs ATTTAGGG
- 2) ADHILKM vs AWHSLPM

Answer

- 1) 22
- 2) 11

The "objective" that the BLAST algorithm tries to maximize is something called the **Maximal Segment Pair (MSP)**, which is the highest-scoring pair of identical-length segments chosen from 2 sequences. For example, consider the following two sequences:

AGCGACTTCAGATCA vs TTCAGATCAGCGCGACCGA

The MSP of this pair of sequences is

AGCGACT**TTCAGATCA** vs **TTCAGATC**AGCGCGACCGA.

Other identical segments include GAC, TTCAG, GCG, etc, but the bolded one is the biggest.

Step 1: Break the query sequence up into "words"

The first step in the BLAST algorithm is to break the sequence you are interested in (aka the "query" sequence) up into "words" (aka sequences of consecutive letters of a particular size). For example, AQTYRCATW is a word of length 9. The words of length 6 in this word are AQTYRC, QTYRCA, TYRCAT, and YRCATW.

? Exercise 1.1.1

How many words of length M are in a word of length N?

Answer

$N-M+1$

Words are often also called "k-mers", where k is the length of the word. Why do we do this? Well, try this next exercise:

? Exercise 1.1.1

Using an alphabet of 4 letters, how many possible words are there of length 5? What about of length 10?

Answer

5: 1024, 10: 1,048,576

It is much easier to search through ~1000 things than it is ~a million!

Step 2: Store the word data for sequences in your database

We now have a set of words from the query sequence. We also have these words for all the sequences in our database. These words can be stored in a table like this (with the example of 4-letter words):

Word (Seed)	Location 1	Location 2	Location 3	Location 4	Location 5	...
AAAA	#1	#2	---	---	#3	---
AAAC	#1	#2	#3	---	---	---
AAAG	---	#1	---	---	#2	---

...	...	...	...	...	...	...
TTTT	#1	...	#2	#3	---	...

The entries in this table (indicated by #1, #2, etc) are the actual locations in the database where the word occurs, and the columns are each possible location in the database. Thus, when you acquire a new sequence, you can break it up into words and add a new column to this table. The words in the first column are called "seeds", because we will use them to start "growing" potential sequence matches.

Step 3: Use seeds to find and extend matches

We can match the words from our sequence (aka seeds) to existing words in the database. Better matches are going to be more similar around these seed matches. Let's take an example sequence TGCATCAA. All 4-letter words from this sequence are: TGCA, GCAT, CATC, ATCA, and TCAA. Each of these words can be used as a seed. For example, the word "GCAT" could be found at the locations #091001, #098199, and #032465. I wrote these locations in this weird way because they represent **pointers**, which are directions to physical locations in the memory of a computer (which is actually how these things are stored). Anyways, we can go to each of these locations and see how the rest of the sequence matches up. Let's say that we have the following sequences at these locations (with the seed word bolded):

- #091001: **GGCAT**CCC
- #098199: AAAAAA**GCAT**AAAAAA
- #032465: **TGCAT**CAA

Note that in general the sequences are going to be much longer than this. For each hit, the basic idea is to start at the seed and then start comparing letters next to the seed. You could go in one direction only, or in both directions. Let's try the first hit, go in both directions, and compute the score at each extension (using the above example where a match is +5 and a mismatch is -4):

- Query: **TGCAT**CAA
- **G****GCAT**CCC: 20
- **GG****GCAT**CCC: 16 (mismatch)
- **GGG****GCAT**CCC: 21 (match)
- **GGGCAT**CCC: 17 (mismatch)
- **GGGCATC**CC: 13 (mismatch)

Notice that continuing to extend the search does not necessarily make the match better. This is because BLAST is a **local** matching algorithm (that's what the L stands for); it wants to find regions of sequences that match and doesn't care if the whole sequence matches or not. This locality is useful when trying to find sequences that may have been diverging for a long period of time but are still similar in some places.

The MSP for this match is the highest score you get, which is 21.

Try the other two potential matches as an exercise:

? Exercise 1.1.1

Compute the MSP score for the targets AAAAAA**GCAT**AAAAAA and **TGCAT**CAA for the query **TGCAT**CAA with seed GCAT.

Answer

22 and 40, respectively

Flavors of BLAST

There are a few different versions of BLAST that are all useful for slightly different things. Regular old BLAST (also called BLASTn) uses nucleotide sequences as queries and targets, and you want to use it if you have a DNA or RNA sequence and want to find the best match to another DNA or RNA sequence. To do the analogous thing with proteins (aka take an amino acid sequence and find the best match amino acid sequence), you use BLASTp. One benefit of BLASTp is you can find proteins that are similar in amino acid structure but could be relatively different in nucleotide sequence.

If you have a nucleotide sequence and don't know the reading frame, you can use BLASTx, which takes your nucleotide sequence, translates it using all 6 reading frames, and then tries to find a best match protein sequence+reading frame. The reverse of BLASTx is called tBLASTn (takes the database nucleotide sequence and translate it to try to match your protein sequence). Finally, if we really have no idea what is going on, tBLASTx translates on both ends (from your nucleotide sequence to potential protein sequences, and from database nucleotide sequences to potential protein sequences).

The table below summarizes these versions (adapted from [here](#)). "Comparison" means what sequences are being compared to compute MSP scores by matching.

Version	Query	Target (Database)	Comparison	Note
blastn	Nucleotide	Nucleotide	Nucleotide-Nucleotide	standard BLAST
blastp	Protein	Protein	Protein-Protein	protein BLAST
blastx	Nucleotide	Protein	Protein-Protein	nucleotide query translated to protein
tblastn	Protein	Nucleotide	Protein-Protein	nucleotide database translated to protein
tblastx	Nucleotide	Nucleotide	Protein-Protein	both the query and database are translated to protein

This page titled [1.1: BLAST \(AKA so you have a sequence\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta](#) (Consortium of Academic and Research Libraries in Illinois (CARLI)) .

1.2: DNA Sequencing

The basic starting point for much bioinformatic analysis is a DNA or protein sequence. In this chapter, we discuss DNA sequencing methods.

Learning Objectives

- Describe the basic methods of first-generation (Sanger), next-generation, and nanopore sequencing
- Compare and contrast the pros and cons of each sequencing method
- Evaluate which sequencing method is most appropriate for example scenarios
- Understand how to interpret raw sequencing data
- Recognize potential errors in sequencing
- Describe quality control methods for post-processing raw sequence data

First Generation Sequencing

The very first successful method of sequencing DNA was developed by Frederick Sanger and others (and is therefore called **Sanger sequencing**). The technique uses a very common principle in molecular biology: if we want to do something to a biological system, let's use how biology does it!

The method is based off of DNA replication, where you have the DNA strands you want to sequence, the DNA polymerase enzyme that replicates the DNA, and a large supply of nucleotides (that will make up the replicated DNA). There are two tricks, though:

1. You also need a small amount of "broken" nucleotides. These nucleotides, once added on the growing replicated DNA, have sort of a "snub end" so that other nucleotides cannot attach onto them. Thus, the replication stops after a broken nucleotide is added. Because this process is random, you get fragments of varying lengths. If you have just the right amount of these "broken" nucleotides compared to the normal nucleotides, you can get sequence fragments of *every* length, as there is a decent probability that some of the DNA replication has stopped at each point.
2. The other trick is that these "broken" nucleotides are labeled (originally radioactively, but now we use fluorescence) in four different "colors", so we know which nucleotide stopped the replication. Thus, we know exactly the identity of the base at that position

Once we have all of these fragments, we run them through a **gel electrophoresis**, which is a technique that separates molecules based on their size. In this case, the size of the DNA is the length of the DNA, so the fragments are ordered from shortest to longest. Because we know what nucleotide is at each length, we can reconstruct the full sequence of the DNA. The below figure shows this process.

Note

Another early method, developed by Allan Maxam and Walter Gilbert, was also developed around the same time as Sanger's method. This alternative method used precise chemistry to break the DNA at specific bonds between specific nucleotides (for instance, you could break the DNA every time there was a C). This method was initially more popular than Sanger sequencing because you could just use the original DNA and didn't have to create many many copies of the original DNA like you do in Sanger sequencing, but overall it is much more technically challenging and is no longer used today.

Reading Sanger Sequencing Output

These days, the output of Sanger sequencing is either an **ab1** file or an **scf** file, which can be viewed in a variety of different software packages. We will take a look at Sanger sequencing output in the R activity.

Sanger sequencing is still commonly used today. It is best used when you are studying only a few genes and want long, accurate read lengths. Another benefit is that the technology used is relatively simple and familiar to many biologists. Once you start needing to sequence many genes, however, Sanger sequencing becomes very expensive and slow. Which brings us to the next generation of sequencing techniques!

Next-Generation Sequencing (NGS)

The most popular "next-generation" style of sequencing was pioneered by people whose company was bought by the company Illumina, hence this type of sequencing is called Illumina sequencing. This method has two basic ideas: make a very large number

of small DNA fragments, each with many identical copies, and then anchor them to a surface. Next, replicate the DNA one base at a time using fluorescently tagged dNTPs, then record the fluorescence for each strand, wash away the dNTPs, and repeat. The benefit of this technique is that you make many identical copies of DNA, so the error rate is small. In addition, you can sequence many different strands of DNA at once. The downside is that the DNA fragments themselves have to be small, so putting them together into a full sequence is challenging and will be the subject of next chapter.

Another NGS method is performed by the company PacBio, hence this type of sequencing is called PacBio sequencing. The basic idea of this method is to observe the replication of the DNA directly. Each fragment of DNA is at the bottom of a small hole, and a small fluorescent detector reads each base that gets attached. The advantage of this method is that the reads are much longer, but the disadvantage is the error rate is a bit higher than Illumina sequencing and it is also a bit more expensive.

Finally, the most modern sequencing technique is called nanopore sequencing, and has been spearheaded by Oxford Nanopore Technologies. The basic idea of this method is to push a DNA strand through a small hole, and measure changes in charge to detect each base. The advantage of this method is that it can create extremely long reads and can be portable, but the error rate is much higher. These various techniques are compared in this table (all numbers are approximate):

Method	Company	Price	Read Length Max	Pros	Cons
Sanger	Various	\$5 per sequence	1kbp	Cheap, accurate	Small datasets
Next-generation sequencing	Illumina, Thermo Fisher	\$200 per genome	300bp	Large datasets (up to whole genome)	More difficult to access
Long-read sequencing	Oxford Nanopore, PacBio	\$1000 per genome	4Mbp	Very long reads	Expensive, slow, somewhat inaccurate

The "cons" for NGS and long-read sequencing are quickly being improved upon by continued technological advances.

Quality Control of Raw NGS Reads

In the sequencing of DNA fragments, each base gets a quality score provided by the sequencing machine based on a calibrated likelihood of mis-identifying a base. These quality scores are also called "Phred" scores. If p is the probability of an error, then the quality score is

$$Q = -10 \times \log_{10}(p). \quad (1.2.1)$$

Thus, $Q = 30$ means that the probability of calling a base incorrectly is 0.001. The higher the value of Q , the better the quality of the base call. Because we want to encode this value in a single character (so that we can put the sequence and the Q score next to each other in a FASTQ file), we use ASCII characters from the following table (obtained from https://www.drive5.com/usearch/manual/quality_score.html).

ASCII_BASE=33 Illumina, Ion Torrent, PacBio and Sanger											
Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII
0	1.00000	33 !	11	0.07943	44 ,	22	0.00631	55 7	33	0.00050	66 B
1	0.79433	34 "	12	0.06310	45 -	23	0.00501	56 8	34	0.00040	67 C
2	0.63096	35 #	13	0.05012	46 .	24	0.00398	57 9	35	0.00032	68 D
3	0.50119	36 \$	14	0.03981	47 /	25	0.00316	58 :	36	0.00025	69 E
4	0.39811	37 %	15	0.03162	48 0	26	0.00251	59 ;	37	0.00020	70 F
5	0.31623	38 &	16	0.02512	49 1	27	0.00200	60 <	38	0.00016	71 G
6	0.25119	39 '	17	0.01995	50 2	28	0.00158	61 =	39	0.00013	72 H
7	0.19953	40 (	18	0.01585	51 3	29	0.00126	62 >	40	0.00010	73 I
8	0.15849	41)	19	0.01259	52 4	30	0.00100	63 ?	41	0.00008	74 J
9	0.12589	42 *	20	0.01000	53 5	31	0.00079	64 @	42	0.00006	75 K
10	0.10000	43 +	21	0.00794	54 6	32	0.00063	65 A			

If you want to eyeball a FASTQ file for error rates, if you see a series of symbols (especially !), then your qualities are low, and you have A-K from the alphabet then the qualities are high.

What do you do with low-quality base calls? Well, if the whole thing is bad we have to re-do the experiment. However, usually we just filter out reads that are not sufficiently high-quality prior to assembly and alignment (the next two chapters).

There are other things we can look at to see if the quality of our reads is good, and most of these are general sanity checks. For example, we can take a look at the frequency of each base; these should remain consistent across all reads at each cycle (aka increase in length). We can also look at **GC content**, which is the fraction of bases that are G or C. This fraction is relatively

consistent within species but can vary quite a bit across species; if your GC content is well outside the expected range for your species, then there might be some contamination.

This page titled [1.2: DNA Sequencing](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta](#) ([Consortium of Academic and Research Libraries in Illinois \(CARLI\)](#)) .

1.3: Genome Assembly

Our DNA sequences are in short fragments. We need to put them together! This process is called **genome assembly**. The key concept for this process is "find the overlaps". If you have two reads that share a sequence, and one read ends with that sequence and the other read begins with that sequence, then you have found a pair of overlapping reads and you can basically assume that they can be combined into one. Let's look at a diagram to see why:

Original sequence

ACAGAGCATCATTACTACGGACA

Reads from sequencing

ACAGAGC

TCATTAC

AGCATCA

CTACGGA

ACGGACA

Reads that overlap in at least 3 bases

ACAGAGC

CTACGGA

AGCATCA

ACGGACA

TCATTAC

Contigs

ACAGAGCATCATTAC

CTACGGACA

CCCCGA

TTAGAC

GACCCCT

GACAGT

CGACCC

In this example, we have sequenced a piece of DNA and made 5 reads. If we (somewhat arbitrarily) set a rule such that we can combine reads that overlap by at least three bases, we have two sets of overlapping reads. When we squash these reads together, we create something called a "contig" (which comes from the same root as "contiguous"). In this case, we were unable to fully assemble the whole sequence because we didn't have enough reads that overlapped with each other and covered the whole sequence, so we are left with two contigs. (This is very common; it is in general very difficult to assemble a whole genome from short reads).

Now, I made that example up and found overlaps by hand. This is not how we do genome assembly! We use networks (or "graphs") to find overlaps between reads, and then we find the most efficient way to combine those reads based on overlaps. Let's start with the "overlap graph", a graph that shows which reads overlap with other reads.

Easing into it with the overlap graph

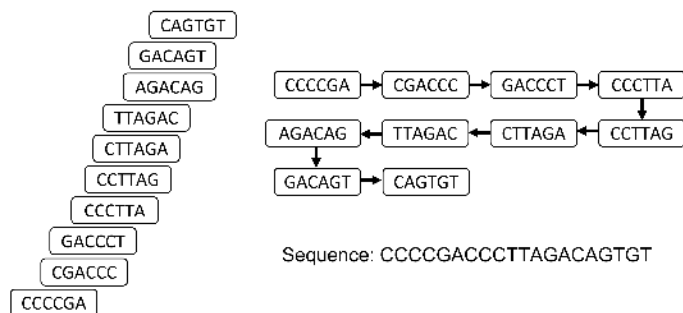
Let's say we have the following reads:

CTTAGA CCTTAG

CAGTGT AGACAG

CCCTTA

Let's construct a graph that connects any reads that overlap in at least 3 bases:



So. That actually worked, because we had enough reads and enough overlap. But! I also did that by eyeballing and by hand, and we need a more rigorous way to do that for more difficult situations.

? Exercise 1.3.1

Construct an overlap graph with minimum 3 overlaps to try to assemble the following reads:

AGCTAG, CACAGCT, ACGCAT, TAGCAT, ATCGAC, CACGCA, GCTAGC

Answer

CACAGCTAGCTAGCAT CACGCAT ATCGAC (you don't have enough reads to cover the whole sequence, so you end up with 3 contigs).

The De Bruijn Graph

In many situations, we use a more complicated technique using a mathematical object called a **De Bruijn graph**. I'll walk you through the whole procedure here.

The first thing to do is: instead of using reads, break down each read into **k-mers** (aka sequences of length k). This way, we can look for overlaps in the middle of reads as well as at the end, account for potential sequencing errors, and not worry about read length. The downside of this is that we lose information about connections that are larger than the k -mers (aka we have to "reconstruct" the read as well as assemble the rest of the genome). There are pros and cons to using this technique instead of the overlap graph, and a lot of it depends on your read lengths, coverage, and computational resources.

? Exercise 1.3.1

Find all the 2-mers, 3-mers, and 4-mers for the read ATGGGCT

Answer

2-mers: AT, TG, GG (2), GC, CT

3-mers: ATG, TGG, GGG, GGC, GCT

4-mers: ATGG, TGGG, GGGC, GGCT

So. Let's say I have a set of reads, and I have acquired the following k -mers from those reads where $k = 3$:

ACC ATA CCC ACG TAG AGA CGA GAC GAT CCT CTA TAG GAC

We're going to do something a bit weird here. We are going to split each 3-mer into 2 2-mers. These 2-mers will be the vertices of the de Bruijn graph. Each original 3-mer will be an edge of the de Bruijn graph, representing the "connection" between the 2 2-mers. So:

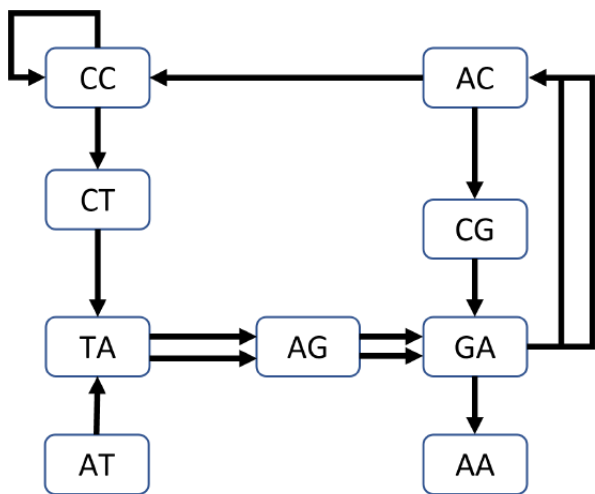
ACC --> AC, CC
ATA --> AT, TA
CCC --> CC, CC
ACG --> AC, CG
TAG --> TA, AG
AGA --> AG, GA
CGA --> CG, GA
GAC --> GA, AC
GAA --> GA, AA
CCT --> CC, CT
CTA --> CT, TA
TAG --> TA, AG
AGA --> AG, GA
GAC --> GA, AC

So, for example, the 3-mer GAC "links" the 2-mers GA and AC. Using this logic, we can construct a graph. Notice how I have kept the repeated 3-mer GAC.

Let's see what this graph looks like:

It's a bit weird-looking, but that's OK. The one special property of the de Bruijn graph that we are going to use is that it is *guaranteed* to have a path where you can hit every single node (aka it is *Hamiltonian*) and edge (aka it is *Eulerian*) exactly once. That path should, in principle, if you have enough coverage from the reads, give you the entire sequence.

Let's see if we can find a path like that in this graph by hand. (Normally we would use a computer). Notice any nodes that have fewer inputs than outputs? AT! So we should probably start there. (We should probably also end at AA, because that has fewer



outputs than inputs).

Start with AT. AT goes to TA, which goes to AG. This blocks out one arrow from TA to AG. AG goes to GA (blocking out one of those two arrows). Now we have a choice. We can either go to AC or AA. If we go to AA, that's a problem, because we can't go anywhere else and we were supposed to hit every node and edge exactly once! So we have to go to AC (blocking out one of those two arrows).

Now we have another choice! We could either go to CC or CG. Let's try CC. Now, CC has a self-loop, meaning we have to go from CC to itself once. So let's do that. Then we have to go to CT, then TA, then AG again, then GA again. Now we have another choice! We still have one arrow left to AC, or we can end it at AA. We have that arrow left, so we have to try to get it. We go to AC. We went to CC last time, so the only other option is CG, then back to GA. Now we can end at AA and relax.

Our path looks like: AT → TA → AG → GA → AC → CC → CC → CT → TA → AG → GA → AC → CG → GA → AA. Squashing this path into a sequence yields: ATAGACCCTAGACGAA.

? Exercise 1.3.1

Try the choice when we are first at AC but decide to go to CG instead of CC. Can you make an Eulerian path? What do you notice is different?

Answer

AT→TA→AG→GA→AC→CG→GA→AC→CC→CC→CT→TA→AG→GA→AA

Yes, but it's a slightly different sequence!

ATAGACGACCCTAGAA. The "GAC" and "CCTA" blocks are switched. Either of these are possible solutions.

You will notice after doing that exercise that there are two possible solutions to this assembly. How can we figure out which one is correct? Well, we usually would try a bigger size of k . Here is the corresponding list of 4-mers, and the corresponding de Bruijn graph.

TAGA CGAT ATAG ACCC GACC AGAC TAGA CCTA CCCT CTAG GACG AGAC ACGA

TAGA → TAG, AGA

CGAT →

CGA, GAA

ATAG →

ATA, TAG

ACCC →

ACC, CCC

GACC →

GAC, ACC

AGAC →

AGA, GAC

TAGA → TAG, AGA

CCTA → CCT, CTA

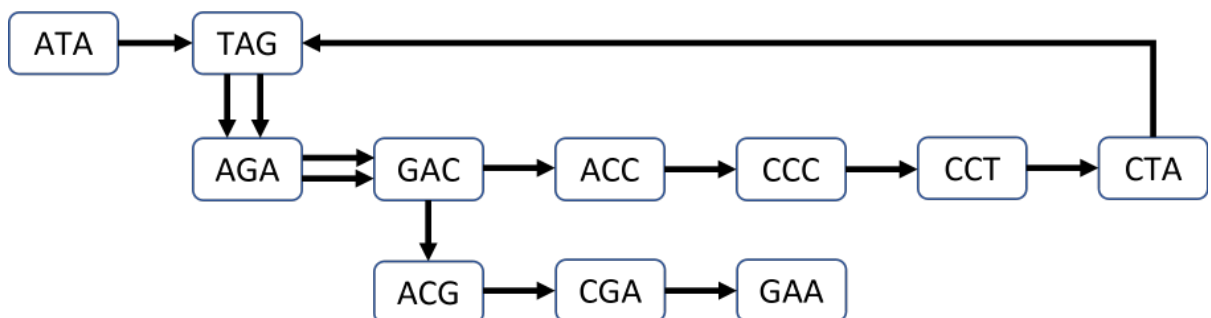
CCCT → CCC, CCT

CTAG → CTA, TAG

GACG → GAC, ACG

AGAC → AGA, GAC

ACGA → ACG, CGA



Now this one is much easier to deal with. You can basically read off the only possible Eulerian path: ATA -> TAG -> AGA -> GAC -> ACC -> CCC -> CCT -> CTA -> TAG -> AGA -> GAC -> ACG -> CGA -> GAA. So our sequence is ATAGACCCTAGACGAA (aka the first choice in the 3-mer version).

Why would anyone choose a smaller value of k ? Well, it is important to remember that a lot of the limitations of bioinformatics problems are computational; sometimes we don't have the resources to do massive computations so we have to make do with smaller ones and hope for the best.

Once you have created contigs from reads by some sort of graph-based method like these, you can partially link these contigs together into **scaffolds**, which are basically contigs connected by a known sequence length but an unknown sequence. This is performed using molecular biology techniques and physical evidence, and has the benefit of adding more structure to your assembly even if you don't know exactly what sequence is connecting the contigs. Assembly programs can attempt to fill in these gaps and figure out the sequences, especially in the case of assembling against a reference. Note that contigs from different chromosomes should never be connected!

How do you evaluate a de-novo assembly?

A **de-novo** assembly is what we have been doing so far, trying to put together a genome from scratch, from just the reads. If you had a reference genome, you could use that reference genome to help you assemble your genome, and the problem becomes much easier. BUt we'll focus on de-novo assembly for this chapter. So, how do you know you did a good job on the assembly?

There are a few different ways to evaluate an assembly, and I'll go through each of them here.

Contig Length Distribution

The number one thing you want out of an assembly is to put as many reads together on a single contiguous segment of DNA as possible. Ideally, we would want one single contig consisting of all the reads, but that is usually impossible unless you have very long reads and/or a very short genome. So, we evaluate our assembly by how well it achieved the target of

1. small number of contigs that are
2. very long and
3. contain as many of the reads as possible

There are a few ways to measure this, and they are all measured by a program called **QUAST**, so I will use that program as an example here.

The most basic measurements are: total # of contigs, the length of the longest contig, the total number of bases in all the contigs, and the total number of bases in the reference genome (if there is a reference). You can also report GC content of the assembly and the reference to see if something weird happened (i.e. you want the GC contents to match). These numbers are mildly useful (you generally want a small # of contigs, a large length of the longest contig, and the total number of bases in all the contigs to be as close as possible to the total number of bases in the reference genome), but consider the following scenarios:

1. two contigs, but almost all reads are unassembled
2. one large contig that contains 30% of the genome, and then 100,000 small contigs

Neither of these are good scenarios, and you wouldn't be able to tell just by looking at the total # of contigs (in the first case) or the length of the longest contig (in the second case). Along with these "absolute" measurements, we need some measurements that look at the whole distribution of contig lengths.

Let's start with an example. Consider the following two read length distributions:

Assembly 1	Cumulative	Cumulative Percentage	Assembly 2	Cumulative	Cumulative Percentage
3383	3383	33.83	1579	1579	15.79
2110	5493	54.93	1458	3037	30.37
2023	7516	75.16	1211	4248	42.48
1023	8539	85.39	1022	5270	52.70
782	9321	93.21	872	6142	61.42

Assembly 1	Cumulative	Cumulative Percentage	Assembly 2	Cumulative	Cumulative Percentage
345	9666	96.66	845	6987	69.87
202	9868	98.68	786	7773	77.73
76	9944	99.44	772	8545	85.45
41	9985	99.85	751	9296	92.96
15	10000	100	704	10000	100

The main two numbers we want to compute here are called N50 and L50. N stands for "number", and L stands for "length". The "50" is the key concept here; we are focused on obtaining 50% of the total genome size. (You can adjust this number to any percentage you like, depending on what standard you want, but the default is 50% People sometimes look at the entire range of possible percentages; this is a bit more of a thorough approach and I would recommend it; my online application shows all of them). The question is: what is the **minimal number of contigs** you need to get 50% of the assembly? (You want this number to be small; you want most of the genome to be captured by as few contigs as possible). The number that answers this question is **L50**. For assembly 1, L50 = 2 (you just need the two longest contigs to hit 50%). For assembly 2, L50 = 4 (you need the 4 longest contigs to hit 50%).

Another question to ask is: what is the **length of the shortest contig** used for 50% of the assembly? This is the length of the shortest contig used when calculating L50. As usual, you want this number to be big. For assembly 1, this length is 2110 bases, and for assembly 2, this length is 1022 bases.

One thing to note about these numbers is that they depend on the length of the assembly. So, if you want to compare across assemblies, it's better to look at 50% of the *estimated genome size* rather than the assembly size (total number of bases in assembly); this version of these statistic are called **LG50** and **NG50**. In our example above, both assemblies have the same size (10,000 bases) so we can compare them directly using L50 and N50.

One infuriating thing is that these two statistics are named backwards! The "N" one measures Length, and the "L" one measures Number! The good news is that L50 is unitless and N50 is in units of bases, so as long as you are careful and report numbers in the appropriate units then it doesn't really matter if you accidentally mix them up.

? Exercise 1.3.1

Which assembly above, 1 or 2, would you prefer to work with? Why? (Don't just say e.g. "because N50 is bigger/smaller"; explain why having a larger or smaller value is better).

Answer

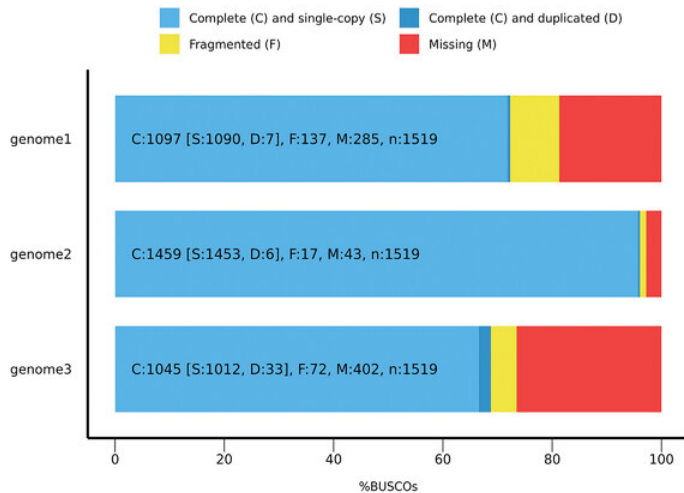
Assembly 1. N50 and L50 are both bigger. For both statistics, we want them to be as large as possible because we want as much information as possible to be on as few contigs as possible. One giant contig and a billion small contigs is probably better than ten evenly-sized contigs, because in the latter case there is no way to combine those contigs together to get the information you get from the giant contig in the first case.

Using Biological Information: BUSCO

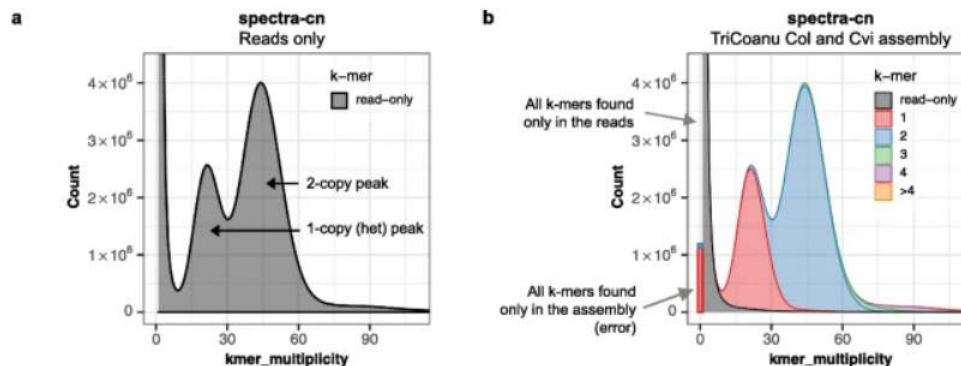
There are genes that are highly conserved across all organisms (aka they are very similar across all organisms, usually because they are so important that any mutation would be bad). One way to see if your assembly is good is if it contains these genes, ideally in one piece. BUSCO (Benchmarking Universal Single-Copy Orthologue) is a method that checks for these genes. In particular, BUSCO looks at single-copy genes that are present in the taxonomic clade of interest (using ortholog data from [here](#)) and tries to find them in your assembly sequence. Here is an example output from a BUSCO analysis (from the [user manual](#)):

These are three example genome assemblies. The x-axis tells you how many of the BUSCO genes were missing (red), present but fragmented (yellow), present, complete, and duplicated (darker blue) and present and in a single-copy (blue). Because these genes are only ever present in a single copy, a duplication indicates an assembly error.

BUSCO Assessment Results



making one genome. In a diploid organism, we would expect most k-mers to appear in the assembly twice, with those appearing once indicating heterozygosity. Ideally, every read will fall into one of these categories, and anything else indicates some sort of assembly error (mapping too many different reads to the same place, perhaps). k-mers that are unique to reads (aka not in the assembly) usually only appear once and probably indicate sequencing error, and k-mers that are unique to the assembly but not in the reads indicate some sort of assembly error at the base level. Below is a plot from the software package [Merqury](#), which performs these kinds of analyses.



locus, then these unique k-mers are probably just sequencing errors.

On the right (b), we count how many times each k-mer comes up in the assembly. Those weird unique k-mers that are probably sequencing errors don't show up at all. The 1-copy and 2-copy kmers appear almost always in the 1-copy and 2-copy spots, so that's good. Some k-mers appear more than twice in the assembly, which indicates an assembly error of some kind, but these are rare. There are also k-mers that are only found in the assembly, which also indicates an error. In general we want almost everything to be within those two main peaks.

Lab Activity

A lab activity can be found at [this GitHub link](#) in the compressed folder "assembly_lab.zip". Inside, there is an RMarkdown file that contains instructions as well as data files.

In general, you want as much as possible to be complete and in a single copy.

k-mer Methods

So far we have looked at the specific contig lengths and also tried to find highly conserved single-copy genes. There is a third "angle" at which we can come at assessing de-novo assembly: checking to see if the assembly itself is consistent with the reads it came from. For these methods, we pick a k, find all k-mers present in the reads, find all k-mers present in the assembly, and compare how many times each k-mer occurs in the reads vs. the assembly.

The count of a particular k-mer in the read set is somewhat irrelevant, as it depends on read depth and there could be many many reads from the same sequence. However, in the *assembly*, the k-mer count is important, because we are

On the left (a) is a histogram of the k-mers looking just at the reads from an NGS experiment. There is a peak for 2-copy k-mers (homozygous) that occur about 45 times each, 1-copy k-mers (heterozygous) that occur about $45/2 = 22.5$ times each, and a peak of k-mers that only occurs about once or twice. If every "proper" k-mer occurs 22.5 times to one particular

This page titled [1.3: Genome Assembly](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta \(Consortium of Academic and Research Libraries in Illinois \(CARLI\)\)](#).

1.4: Genome Alignment

Suppose you have a sequence, or at least a series of contigs. What do you do next? One thing you can do is **align** that sequence to a **reference genome** that you already know a lot about. Aligning a sequence to another one is basically just matching the two to each other. There are many reasons you would want to do this, including finding similar genes, finding homologous genes (similarity by evolutionary history), comparing genomes, and assembling reads using a reference.

We'll start by defining what a "match" is, then we'll go through three algorithms that we use to align genomes.

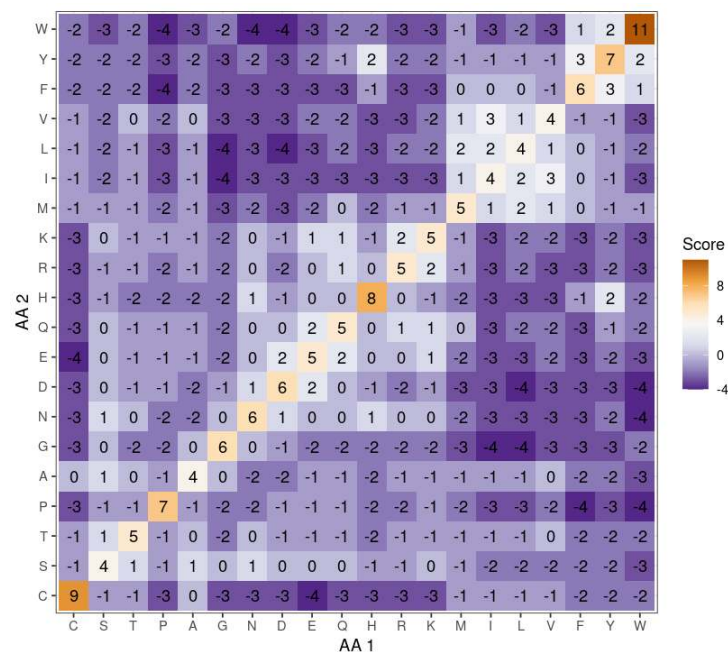
PAM Matrix

So what is a "match"? How different are different parts of a sequence?

If we are doing, say, a Google search, and we want an exact match, we put quotes around the query (e.g. "alignment to a reference genome" will search for this exact sentence). By contrast, if you just put that phrase in without quotes, you can get those words mostly in the same order, but possibly with other words in between. When we align genomes, we want to leave room for gaps and mutations, so the process is more similar to the second kind of search.

If we have a DNA sequence, the simplest model to use is to treat every mismatch as equal. So e.g. A vs T is the same as A vs G. However! Some mutations are more likely than others, so if you wanted to be a bit more accurate you'd treat the less likely mutations as being "further apart". We'll go over these mutation rates more in the chapter on phylogenetics.

When we align sequences, we usually want to use the protein (amino acid) sequence, as there's more information there (we can ignore silent mutations). Of course, when we build a tree and do phylogenetics, we want to use the DNA sequence because silent mutations still provide useful information about phylogeny, but here we're just trying to line up some strings. Now, amino acid changes are way more complicated than nucleotide changes. What constitutes a "similar" amino acid, or a "likely" change, or an "unlikely" change? The first attempt at creating a matrix that tells you how much to "score" a match or mismatch based on sequence similarity was performed by the founder of bioinformatics, Margaret Dayhoff (see Box BOX). This matrix was called the PAM (point-assisted mutation matrix), and was obtained by looking at different sequences of highly-conserved proteins and seeing how often various amino acid changes happen. The original PAM matrix was used for a while, but has recently been superseded by a matrix called the BLOSUM matrix, which uses sequences that slightly more diverged and can be more accurate over less closely-related sequences.



How you use this matrix is: line up your two sequences, and compare the two amino acids at each position. Add up the scores you get from each comparison. For the BLOSUM matrix, a higher score is better (aka most matching). These numbers are log-odds: you take the probability that you get a particular pair (say, T in one sequence and M in the other sequence) and divide that by the

probability that you would get that pair just by randomly pairing up the two sequences (so if T and M were very frequent, you'd be likely to get that pair randomly anyways). A positive number means you are more likely to get that pair by chance, and a negative number means you are less likely to get that pair by chance.

? Exercise 1.4.1

1. First, let's consider the diagonal. AKA, if you see a "C" in both sequences, you add +9. If you see a "Q" in both sequences, you add +5. Which amino acid are you *less likely* to see pair with itself by chance? That is, between C and Q, which one tends to have more mutations away from it?
2. Name the *most likely* substitutions and the *least likely* substitutions.

Answer

1. Q; the log-odds of seeing QQ is lower than seeing CC
2. The highest positive off-diagonal value is 3, which is the I/V substitution. The lowest negative value is -4, which occurs for W/P, W/N, W/D, F/P, L/G, L/D, I/G, and E/C.

Needleman-Wunsch Algorithm

Now that we have a way of scoring an alignment to see how good it is, we need to make that alignment in the first place! As with everything else, "eyeballing" isn't good enough; we need an algorithm to use. We'll start with the Needleman-Wunsch algorithm, which is used for **global alignment** (aka let's align the entire sequences together with gaps if necessary; this is opposed to **local alignment** where we care about aligning *some* part of each sequence together and may not care about the whole thing).

We start with two sequences we want to align:

AGCTACAT
ACTAGCAT

In general, we would use a fancy scoring matrix like a PAM or BLOSUM matrix, but for this demonstration we'll use a simple scoring system: a match is +1, a mismatch is -1, and a gap is -2.

We start by making a big grid with one sequence on top and the other sequence on the side. We have a blank label for the first row and first column, and always start by putting a "0" in the top-left.

		A	G	C	T	A	C	A	T
A	0								
C									
T									
A									
G									
C									
A									
T									

Now, each cell is going to be filled out by row, so you start from the top left and move left-to-right. For each cell, you take the maximum of the following three values:

1. The score **above** minus the gap penalty
2. The score to the **left** minus the gap penalty
3. The score to the **diagonal top-left** plus the match score if this row and column is a match or minus the mismatch penalty if this row and column is a mismatch

In an alignment with no gaps, all we are doing is comparing matches and mismatches, and we'd be going down the diagonal and using only the 3rd number. However, this algorithm allows us to incorporate gaps by shifting either up/down or left/right (for a gap) and then continuing down diagonally bottom-right as we continue matching up the alignment.

Filling out the first row this way is different from everything else because there is no "above" row so we just use calculation 2. Similarly, for the first column, there is no "left" column so we just use calculation 1. In both these cases, we just subtract the gap penalty over and over.

OK, now let's start at the next thing to be filled in, which is the top-left empty space. Calculation 1 is "-2 - 2 = -4", where the first -2 is the top score, and the second -2 is the gap penalty. Calculation 2 is also "-2 - 2 = -4", where the first -2 is the left score. Calculation 3 is "0 + 1 = 1" because it's a match (A and A). The maximum between -4, -4, and 1 is 1, so we fill that in. Notice that in our scoring system, gaps are worse than mismatches, so when we do the full alignment we may allow more mismatches than gaps. In general, you can change your scoring system to weigh gaps more or less heavily than mismatches, depending on how many gaps you might expect in your alignment.

Now we move one cell to the right. Calculation 1 is "-4 - 2 = -6". Calculation 2 is "-2 - 2 = -4". Calculation 3 is "-2 - 1 = -3" because G and A are a mismatch. The maximum of -6, -4, and -3 is -3, so we fill in -3.

		A	G	C	T	A	C	A	T
	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2								
C	-4								
T	-6								
A	-8								
G	-10								
C	-12								
A	-14								
T	-16								

		A	G	C	T	A	C	A	T
	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	1	-3						
C	-4								
T	-6								
A	-8								
G	-10								
C	-12								
A	-14								
T	-16								

I have filled the rest of the table in.

		A	G	C	T	A	C	A	T
	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	1	-1	-3	-5	-7	-9	-11	-13
C	-4	-1	0	0	-2	-4	-6	-8	-10
T	-6	-3	-2	-1	1	-1	-3	-5	-7
A	-8	-5	-4	-3	-1	2	0	-2	-4
G	-10	-7	-4	-5	-3	0	1	-1	-3
C	-12	-9	-6	-5	-5	-2	1	0	-2
A	-14	-11	-8	-7	-6	-4	-1	2	0
T	-16	-13	-10	-7	-6	-6	-3	0	3

From this table, we find the optimal alignment by starting at the bottom left and then tracing a path through this table. At each cell, find the highest number from either its top neighbor, left neighbor, or top-left diagonal neighbor, and then go there. If multiple numbers of those three are equal, those represent equally valid optimal alignments, so you have to go down those different paths separately. If you choose the diagonal, then write down both of the corresponding bases/amino acids. If you choose to go up, that means its a gap in the side sequence aligned with the letter in the top sequence, and if you choose to go left, that means its a gap in the top sequence aligned to the letter in the side sequence. So. Let's do this for our example.

		A	G	C	T	A	C	A	T
	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	1	-1	-3	-5	-7	-9	-11	-13
C	-4	-1	0	0	-2	-4	-6	-8	-10
T	-6	-3	-2	-1	1	-1	-3	-5	-7
A	-8	-5	-4	-3	-1	2	0	-2	-4
G	-10	-7	-4	-5	-3	0	1	-1	-3
C	-12	-9	-6	-5	-5	-2	1	0	-2
A	-14	-11	-8	-7	-6	-4	-1	2	0
T	-16	-13	-10	-7	-6	-6	-3	0	3

The alignment that you can read off of this table (remember when you write this down you start at the end and go backwards) is:

Top: T -> AT -> CAT -> -CAT ->
 A-CAT -> TA-CAT -> CTA-CAT ->
 GCTA-CAT -> AGCTA-CAT
 Side: T -> AT -> CAT -> GCAT ->
 AGCAT -> TAGCAT -> CTAGCAT -> -
 CTAGCAT -> A-CTAGCAT

Was that the alignment you would have come up with just by eyeballing? Probably. But we use this algorithm for much longer sequences where things are not as straightforward.

The Needleman-Wunsch algorithm is used for global alignment; we want to align the whole sequences together as well as possible. If we were concerned about **local alignment** (aka we know that most of the sequence will not align very well but we hope that some portion of it does), then the algorithm we use is a slightly modified version called the **Smith-Waterman algorithm**.

The main differences between the Needleman-Wunsch (NW) and Smith-Waterman (SW) algorithms are:

1. No negative numbers for SW. You can imagine that in the NW algorithm, large regions of mismatch would lead to a buildup of negative numbers that is a "hole" that partially-aligning parts of the sequence won't be able to "dig out of", and so sequences that align poorly overall are going to have horrible scores even though they align well in certain specific places. The SW algorithm makes all numbers that would be negative in NW zero, so that you can kind of "start fresh" every time there is a bad set of mismatches. The way this is implemented is that for NW, there were three numbers you had to find the maximum of, but in SW if that number ends up being negative you force it to be 0.
2. Instead of starting at the bottom-right and ending at the top-left (aka aligning the whole sequence), you start at the highest number and end once you hit 0 (aka you are trying to find the best partial (aka local) alignment between the two sequences).

Here I have performed the SW algorithm instead of the NW algorithm on this example.

Notice that there are few places we could have started if that last "T" wasn't there (one is highlighted in bright green; the other two are part of the solution), and these correspond to places where the sequences align locally but the matching parts are in different places overall (e.g. "AGC" is in both, and "CTA" is in both). But because the last "T" has that "4", we have to start there.

	A	G	C	T	A	C	A	T
	0	0	0	0	0	0	0	0
A	0	1	0	0	0	1	0	1
C	0	0	0	1	0	0	2	0
T	0	0	0	0	2	0	0	1
A	0	1	0	0	0	3	1	1
G	0	0	2	0	0	1	2	0
C	0	0	0	3	1	0	2	1
A	0	1	0	1	2	2	0	3
T	0	0	0	0	2	1	1	4

The optimal local alignment here is

Top: T -> AT -> CAT -> -CAT -> A-CAT -> TA-CAT -> CTA-CAT

Side: T -> AT -> CAT -> GCAT -> AGCAT -> TAGCAT -> CTAGCAT

Almost the same as the NW result, but we stopped before aligning first two bases of the top sequence or the first base of the bottom sequence. That's because our gap penalty is high compared to our mismatch penalty. A high gap penalty will lead to a lot of zeros, so you'll end up stopping instead of trying to put a gap in more frequently.

Here is the same alignment, but with the gap and match/mismatch penalties switched in magnitude (so gaps are -1, matches are +2, and mismatches are -2).

	A	G	C	T	A	C	A	T
	0	0	0	0	0	0	0	0
A	0	2	-1	0	0	2	1	2
C	0	1	0	3	2	1	4	3
T	0	0	0	2	5	4	3	2
A	0	2	1	1	4	7	6	5
G	0	1	4	3	3	6	5	4
C	0	0	3	6	5	5	8	7
A	0	2	2	5	4	7	7	10
T	0	1	1	4	7	6	9	12

This alignment looks like:

Top: T -> AT -> CAT -> ACAT -> -ACAT -> T-ACAT -> CT-ACAT -> GCT-ACAT -> AGCT-ACAT

Side: T -> AT -> CAT -> GCAT -> AGCAT -> TAGCAT -> CTAGCAT -> -CTAGCAT -> A-CTAGCAT

So this is a different result! We managed to align the whole thing, but this time something different happened at around the 5th base. So this example

demonstrates that sometimes it does matter what penalties and scores you use.

? Exercise 1.4.1

Use the Needleman-Wunsch Algorithm to align the following sequences:

ACGTGA

AGTGA

Using: matches = +2, mismatches = -1, and gaps = -2.

Answer

	-	A	G	T	G	A
-	0	-2	-4	-6	-8	-10
A	-2	2	0	-2	-4	-6
C	-4	0	1	-1	-3	-5
G	-6	-2	2	0	1	-1
T	-8	-4	0	4	2	0
G	-10	-6	-2	2	6	4
A	-12	-8	-4	0	4	8

ACGTGA

A-GTGA

Multiple Sequence Alignment

So far we have only been talking about how to align two sequences to each other (aka pairwise alignment). Frequently, however, we need to align multiple sequences together simultaneously. One use for this is in building phylogenetic trees: you need to know how different the sequences are, and for that you need to directly compare homologous sequences across multiple samples. We could align every pair of samples separately and compute some sort of genetic distance between the two samples, and use a distance-based phylogenetic method. However! There is a lot of potential information in the other samples that you could use to help align them all better with each other, doing all pairs is computationally inefficient, and there are better phylogenetic methods than distance-based ones that require the simultaneous alignment of multiple samples. So let's see how we do this.

The most common technique for doing a Multiple Sequence Alignment (MSA) is via *progressive alignment*. This is an iterative process that starts with unaligned sequences and creates a pairwise distance matrix between them, usually starting with a pairwise alignment method that I described above. This distance matrix is used to make a distance-based tree that is used to divide the alignment problem into smaller problems; this tree is called a "guide tree". Then, small groups of sequences that are clustered together on the guide tree are aligned separately, and once those alignments are complete, the groups are aligned with each other.

There are many different algorithms you can use for an MSA, and some of them are better in different circumstances. The most popular right now are MAFFT, MUSCLE, and ClustalOmega. Here is an example of an MSA produced by ClustalX (an older version of Clustal):

```

      *
      :
      :
Q5E940_BOVIN -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_HUMAN -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_MOUSE -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_RAT -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_CHICK -----MPREDRATWKSNYFMKIIQLDDYPKCFVVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_RANSY -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--SALE 76
Q7ZUG3_BRARE -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_ICTPU -----MPREDRATWKSNYFLKIIQLDDYPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_DROME -----MVRENKAQWKAQYFIVKVFLEDFEPKCFIVGADNVGSKMQQIRMSLRGK-AVVLGKNTMMRKAIRGHLENN--PALE 76
RLA0_DICDI -----MSGAG-SKRRKLFIEKATKLTFTYDKMIVAEDFVGSSQLQKIRKSRIGI-GAVLMGKNTMIRKIVIRDLADSK--PELD 75
Q54LP0_DICDI -----MSGAG-SKRRNVFIEKATKLTFTYDKMIVAEDFVGSSQLQKIRKSRIGI-GAVLMGKNTMIRKIVIRDLADSK--PELD 75
RLA0_PLAF8 -----MAKLSKQKKQMYIEKLSSLIQQYSKLIVHVDNVGSKMQQIRMSLRGK-ATILMGKNTIRIRALKKNLQAV--DQIE 76
RLA0_SULAC -----MIGLAVTTTKKIAKWKVDEVAELTEKLKTHKTIIANIEGFPADKLHEIRKKLRGK-ADIKVTKNLNFNIALKNAG----YDTK 79
RLA0_SULTO -----MRIMAVITQERKIAKWKIEEVKELEKLRHTTIIANIEGFPADKLHDIRKKMRGM-AEIKVTKNLTFGIAAKNAG----LDVS 80
RLA0_SULSO -----MKRLALALKQKQKVASWKEEVKELTELKKSNTILICNLEGFADKLHEIRKKLRGK-ATIKVTKNLTFKIAAKNAG----IDIE 80
RLA0_AERPE MSVVSIVGQMYKREKPIPEWKTLMLELEELFSKHVFLFADLTGPTFVVRVRKKLWKK-YPMVAKKRIILRAMKAAGLE--LDDN 86
RLA0_PYRAE -MMLAIGKRRYVTRQYIPARKVKIVSEATELLQKYPYVFLFDLHGLSSRIIHEVYRLRY-CVIKTIKPLFKIAFTKVYGG--TPAE 85
RLA0_METAC -----MAEERHHTHEHIPQWKDEIENIKELIQSHKVFQVGVIEGILATKIKIRRDLDKV-AVLKVSRRNTLTERALNQLG--ETIP 78
RLA0_METMA -----MAEERHHTHEHIPQWKDEIENIKELIQSHKVFQVGVIEGILATKIKIRRDLDKV-AVLKVSRRNTLTERALNQLG--ESIP 78
RLA0_ARCFU -----MAAVRGS--PEYKVRAVEEIKRMISSKPVVAIVSFRNPAGQMKIRREFRGK-AEIKVVKNTLLERDALG--GDYL 75
RLA0_METKA MAVKAKGQPPSGYEAKVAEWKRREVKELKELMDEVNGLVDLEGIPAPQLQEIIRAKLRERDIIRMSRRNTLMRIALEEKLDER--PELE 88
RLA0_METTH -----MAHVAEWKKKEVQELHDLIKGYEVVGIANLADIPAROLQKMRQTLRDS-ALIRMSKKLLISLALEKAGREL--ENVD 74
RLA0_METTL -----MITAESEHKIAPWKIEEVNKLKELLKNGQIIVALVDMMEVPAROLQEIIRDKIR-GTMTLKMSRRNTLIERAIEVAEETGNPEFA 82
RLA0_METVA -----MIDAKSEHKIAPWKIEEVNALKELLKSNVIALIDMMEVPAROLQEIIRDKIR-DQMTLKMSRRNTLIERAIEVAEETGNPEFA 82
RLA0_METJA -----METKVKAHVAWKIEEVKTLKGLIKSKPVVAIVDMMDVPAPQLQEIIRDKIR-DKVKLRMSRRNTLIERAIEVAEELNNPKLA 81
RLA0_PYRAB -----MAHVAEWKKKEVEELANLKSYPVIALVDVSSMPAYPLSQMRRLIRENGGLLRVSRNTLIERAIEVAEELGKPELE 77
RLA0_PYRHO -----MAHVAEWKKKEVEELAKLKSYPVIALVDVSSMPAYPLSQMRRLIRENGGLLRVSRNTLIERAIEVAEELGKPELE 77
RLA0_PYRFU -----MAHVAEWKKKEVEELANLKSYPVIALVDVSSMPAYPLSQMRRLIRENGGLLRVSRNTLIERAIEVAEELGKPELE 77
RLA0_PYRKO -----MAHVAEWKKKEVEELANLKSYPVIALVDVAGVPAYPLSKMRDKLE-GKALLRVSRNTLIERAIEVAEELGKPELE 76
RLA0_HALMA -----MSAESERKTEIPEWKQEEVDIVEMIESYESVGVVNIAGIPSRQLQDMRRDLHGT-AELRVSRNTLIERALDDVD--DGLE 79
RLA0_HALVO -----MSESEVRQTEVPEWKREVEDELVDVIESYESVGVVGVAGIPSRQLQDMRRDLHGS-AAVRMSRRNTLVNRALEVN--DGFE 79
RLA0_HALSA -----MSAEQRTEVPEWKREVEDELVDLETYSVGVVNTGIPSKQLQDMRRDLHGO-AALRMSRRNTLVRALEEAG--DGLD 79
RLA0_THEAC -----MKEVSSQKKELVNEITIRIKASRSVAIVDLAGIRTRQIQDIRGKNGRGK-INLVKIKLILFKALENLGD--EKLS 72
RLA0_THEVO -----MRKINPKKKEIVSELADITKSKAIVAVDKGVTRQMODIRAKNRDK-VKIKVKKKLLFKALDSIND--EKLT 72
RLA0_PICTO -----MTEPAQWKIDFVKNLLENINSRKVAIVSIKGLRNNFQKIRNSIRDK-ARIKVSRRALLRLAIENTGK--NNIV 72
ruler 1.....10.....20.....30.....40.....50.....60.....70.....80.....90

```

(By Miguel Andrade at English Wikipedia - Transferred from en.wikipedia to Commons., CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=3930704>)

Each row is a sample sequence, and each column is a site. In this case, we are looking a protein alignment, so the sites are amino acids. A dash indicates a gap. Colors can indicate many different things in a MSA; in this case they are related to the properties of the amino acid (basically, the same color means the same amino acid, and a color is shown if the amino acid is highly conserved across sequences and share similar properties). By visualizing alignments like this, we can see how successful they are and quickly spot mutations. Amino acid properties include things like polarity/hydrophilicity.

? Exercise 1.4.1

In the above alignment:

1. What do you think the asterisk (*) stands for?

2. What is the most common length of this protein?
3. Between what 20bp stretch between 10bp markers is the protein most conserved? What about most variable?

Answer

1. All samples have the same amino acid
2. 76 aa
3. 60-80, 1-20

Lab Activity

A lab activity called "alignmentlab.pdf" is part of the supplemental files for this textbook. An Rmarkdown file of this lab can be found at [this link](#).

This page titled [1.4: Genome Alignment](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta](#) ([Consortium of Academic and Research Libraries in Illinois \(CARLI\)](#)).

1.5: Gene Annotation

What is gene annotation?

Once you successfully obtain a DNA sequence, one of the things you might want to find out is "what is this sequence?" The process of figuring out what a particular sequence of DNA actually *is* is called **annotation**. When we're trying to see if something is a protein-coding gene, we perform **gene annotation**.

There are two kinds of annotation: **structural annotation** --- where we divide the potential gene up into regions that tell us something about the gene's structure (such as where the start/stop codons are, where the transcription factor binds, etc), and **functional annotation** --- where we try to figure out what the gene *does* (aka what the protein that the gene creates does). We'll start with structural annotation.

Structural Annotation

Finding start and stop codons

The most basic thing you can do with a DNA sequence is to find any potential start and stop codons, and then try to determine potential open reading frames (ORFs). As a reminder, a **start codon** is a three-letter sequence of DNA which is where transcription starts, and a **stop codon** is a three-letter sequence of DNA which is where transcription stops. In almost all eukaryotes and many bacteria and archaea, the start codon is **ATG**. The stop codons are **TAA**, **TAG**, and **TGA**.

Fun Fact: Different genetic codes

Not all organisms use the same genetic code that we do! The NCBI has a list of all known genetic codes [here](#). Most of the differences are not in the start and stop codons. Almost all eukaryotes (organisms with a membrane-bound nucleus like us) use the same start codon, although there are a couple of possible rare alternatives. Archaea are a little bit more flexible, but they share common transcriptional machinery with us so they'll be more similar to us than they are to Bacteria. Bacteria can vary their start codon much more freely. Thus, when performing annotation on prokaryotic genomes, you need to account for the fact that there are multiple possible start codons and that some are more likely to be used than others.

Historical Note

The three stop codons have fun names to distinguish them! They are called **amber (TAG)**, **ochre (TAA)**, and **opal (TGA)**. The first one to be discovered was called **amber** after the English translation of the last name of a then-graduate student, [Harris Bernstein](#), who did most of the experimental work.[1] The next two were named after other colored minerals in keeping with the theme. This article from Scientific American has a nice historical background of the work that isolated these codons, written by the people who did the early work on it:

Edgar, R. S., and R. H. Epstein. "The Genetics of a Bacterial Virus." *Scientific American*, vol. 212, no. 2, 1965, pp. 70–79. *JSTOR*, <https://www.jstor.org/stable/24931779>. Accessed 4 Sept. 2024.

The general method for finding a small sequence in a bigger sequence where there is some possibility of variation in the small sequence is by using a **position-specific scoring matrix (PSSM)**, also called a position-weight matrix (PWM) or position-specific weight matrix (PSWM). We will try to find all the start and stop codons in an example DNA sequence using this method in a few different ways. Now, most of the time you can just eyeball a sequence to find potential start and stop codons, or literally just search for the specific sets of three letters, but this is a good place to start learning about the more general method.

Start Codon (ATG)

Let's start with this sequence:

TATGCGTTGATTCGTCTAAC

? Exercise 1.5.1

Eyeball the above sequence to find the positions of possible start and stop codons.

Answer

The start codon is at position 2, and there are two possible stop codons at positions 8 and 17.

Now let's construct a PSSM and do this formally. The way we construct a matrix like this is we take all the possible sequences that we're looking for and see how frequently each base comes up at each position in the sequence. For the start codon, this is easy. It's **A** 100% of the time, then **T** 100% of the time, then **G** 100% of the time. We represent this in matrix form by:

Base	Position 1	Position 2	Position 3
A	1	0	0
C	0	0	0
G	0	0	1
T	0	1	0

? Exercise 1.5.2

Create a PSSM for the stop codon **TAA**

Answer

Base	Position 1	Position 2	Position 3
A	0	1	1
C	0	0	0
G	0	0	0
T	1	0	0

📌 Note

Suppose I tried to create a PSSM for "the stop codon" and came up with something like this:

Base	Position 1	Position 2	Position 3
A	0	0.67	0.67
C	0	0	0
G	0	0.33	0.33
T	1	0	0

There are multiple reasons why this is incorrect. The first is that I treated all three stop codons equally, when in reality some are more common than others. The most important reason, however, is that we *know* what the stop codons are *exactly*, and there's no reason to make this probabilistic. Any time we see either a TGA, TAG, or TAA, that's *always* a stop codon, and *nothing else* is. PSSMs assume *independence* of each position, and in the case of these stop codons this is **not** true.

Exercise 1.5.3

There is one three-letter code that has positive probability using this table but is **not** a stop codon. Which one is that? Why does the independence assumption of PSSMs lead us to do something wrong here?

Answer

TGG. It would have a lower probability than the actual stop codons, but it still shows up as possible using this matrix. This is because positions 2 and 3 are **not actually** independent of each other in our stop codons. The first position is always a T, but the next two can either be A or G. However! Only the combinations AG, AA, and GA are possible; GG is not! Thus, the two positions are not independent of each other, and making a PSSM like this is incorrect.

To use a PSSM, we pick a potential starting site and use the matrix to get probabilities for each letter in the potential sequence. Let's start with the first position:

TATGCGTTGATTTCGTCTAAC

The probability of T in the first position is 0, A in the second position is 0, and T in the third position is 0. So the overall probability that this triplet is a start codon is

$$0 \times 0 \times 0 = 0 \quad (1.5.1)$$

If we then try, say, position 2, we get

TATGCGTTGATTTCGTCTAAC

The probability of A in the first position is 1, T in the second position is 1, and T in the third position is 1. So the overall probability that this triplet is a start codon is

$$1 \times 1 \times 1 = 1 \quad (1.5.2)$$

Repeating this for all the positions yields all 0s except for position 2. We have found 1 start codon (at position 2) using this method. Admittedly, this method is overkill for finding start and stop codons; generally we just search for all exact matches of the three letters. We will use PSSMs to find much more difficult-to-detect sequences in section SECTION.

Open Reading Frames

The sequence between a start and a stop codon is called an **open reading frame** or ORF; this is the part that is actually transcribed and turned into a protein eventually. In our example, there are two possible ORFs:

TATGCGT**TGA**TTTCGTCTAAC

and

T**TATGCGTTGATTTCGTCTAAC**

In prokaryotes, there are no introns, so the DNA sequence between the start and stop codons must be multiples of three long. In eukaryotes, we can have introns, so we cannot eliminate any potential ORFs based on length like that. But! In this case we can eliminate the second possibility for a simple reason: it has a stop codon in the middle! An ORF cannot have an in-frame stop codon in the middle; the transcription will never get through the whole sequence.

Activity 1 uses an online tool to explore finding ORFs in short sequence data. R activity 1 provides an example of annotating a full eukaryotic gene.

Regulatory Sequences

Let's look at an example of how to use a PSSM to detect a specific kind of regulatory sequence. The **Kozak consensus sequence** is a sequence around the start codon that is thought to optimize the starting of translation in eukaryotes. More information on Marilyn Kozak can be found in the HISTORICAL BOX. In her original paper in 1987, Marilyn Kozak tabulated the nucleotide distributions around 699 mRNA sequences from vertebrates. Here is the PSSM matrix, adapted from that paper:

Base	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16

Base	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16
A	0.23	0.26	0.25	0.23	0.19	0.23	0.17	0.18	0.25	0.61	0.27	0.15	1	0	0	0.23
C	0.35	0.35	0.35	0.26	0.39	0.37	0.19	0.39	0.53	0.02	0.49	0.55	0	0	0	0.16
G	0.23	0.21	0.22	0.33	0.23	0.20	0.44	0.23	0.15	0.36	0.13	0.21	0	0	1	0.46
T	0.19	0.18	0.18	0.18	0.19	0.20	0.20	0.20	0.07	0.01	0.11	0.09	0	1	0	0.15

The sequence with the highest probability for each position is the **consensus sequence**: CCC GCC GCC ACC [ATG] G, where the start codon is in brackets.

? Exercise 1.5.4

Use the PSSM to determine which of the following two sequences is more likely to be a translation initiation site:

CTC GAG GCC AGC [ATG] G

AAA GCT GCT ACC [ATG] G

Answer

The first one has total probability

$$0.35 \times 0.18 \times 0.35 \times 0.33 \times 0.19 \times 0.20 \times 0.44 \times 0.39 \times 0.53 \times 0.61 \times 0.13 \times 0.55 = 1.096819 \times 10^{-6} \quad (1.5.3)$$

. The second one has total probability

$$0.23 \times 0.26 \times 0.25 \times 0.33 \times 0.39 \times 0.20 \times 0.44 \times 0.39 \times 0.07 \times 0.61 \times 0.49 \times 0.55 \times 0.46 = 3.495518 \times 10^{-7} \quad (1.5.4)$$

, so the first one is more likely to be a translation initiation site.

Other kinds of regulatory sequences include TATA boxes, Pribnow sequences, Shine-Dalgarno sequence (for ribosome binding sites), and others.

📌 Note: What if there's a zero probability?

In the Kozak consensus sequence example above, positions 13-15 had some bases at potentially zero probability. In this case, this is because we know that start codons must have a specific sequence. But what if there is in fact potential variation in the sequences that we just haven't observed in samples yet? If we set those to zero, we could miss out on detecting something correctly with a different variant than what we have already observed. This might be unlikely, but surely it isn't impossible!

The way we generally get around this problem is to use "pseudocounts" instead of just counting the base pairs that show up at a particular position. So, instead of counting B_i , the number of base pair B at position i , and dividing by N , the number of samples, to get the frequency of that base at that position, we do the following:

Take $\sqrt{N}$ and multiply it by the overall frequency of base B . Add that number to B_i , and divide that sum by $N + \sqrt{N}$. That's your new frequency. This may look weird, but it comes from [Bayesian statistics](#).

In addition to using pseudocounts, you might have noticed that the probabilities in exercise 4 are very small. What we typically do instead of multiplying all those probabilities is taking the negative log of each probability and then adding them together.

📌 Note: State-of-the-art

The techniques we go over in this chapter are fairly basic. The technique that is currently used by structural annotation software uses **Hidden Markov Models** (HMMs), which are beyond the scope of this textbook. For now, [this tutorial](#) is a good place to start.

Introns and Exons

Exons (aka the protein-coding regions of a gene) are typically bookended by specific sequences. An exon can start right after the start codon, or it can start at a 3' "acceptor" site after an intron. This acceptor site typically has the form CAG/G, where nucleotides upstream of the slash (which is the cut site) are in the intron and nucleotides downstream of the cut site are part of the exon. The most highly-conserved part of this pattern is the "AG" before the cut site, which almost never changes.

Exons can end either just before stop codon or at a 5' "donor" site, which typically has the sequence GG/GT (or GG/GU for RNA). In this case, nucleotides upstream of the slash are part of the exon, and nucleotides downstream are part of an intron. The most highly-conserved part of this pattern is the "GT" after the cut site, which almost never changes.

Because these distinctive sequences are so short, it's not super useful make a PSSM for them. Most annotation techniques today use Hidden Markov Models (HMMs), which have much more complicated models of predicting when something is an exon or an intron.

? Exercise 1.5.5

What is an alternative way to find out if something is an exon without even necessarily looking at the sequence specifically? (Hint: what does an exon do that an intron doesn't)?

Answer

Introns are spliced out of mRNA and the exons are kept in. So, just sequence post-splicing mRNA and align it to the DNA. The exons will match the mRNA, and the introns won't match anything! This technique is a common way to verify if something is an exon. It cannot, however, be used to say that something is *not* an exon, because many genes are turned off or downregulated in different tissues or at different times and don't actually produce mRNA all the time.

Other Types of Sequences

There are many parts of a genome that are not protein-coding. For instance, some parts of the genome code for tRNA and rRNA. These sequences are very highly conserved and their patterns are recognizable enough that we have specific tools to identify them that work extremely well. So, typically, when you want to annotate a genome, one of the first things you do is run one of these tools on the genome to identify tRNA- and rRNA-coding DNA, and then ignore those for the rest of the annotation process.

Bacteria, like everything else, are constantly under attack from viruses. These viruses (called "bacteriophages") often leave pieces of their genome inside the bacterial genome, and these sequences also have specific patterns that we can look for. (We can also, like with tRNA and rRNA sequences, compare them to a relatively small database of known sequences). So when annotating bacterial genomes, an early step is often to look for these "prophage" genes.

Eukaryotes have a lot of repetitive DNA. Some of this DNA, called short tandem repeats (STRs) or "microsatellites", consists of short sequences repeated over and over. These are relatively easy to identify and then "mask" (or ignore) for the rest of the annotation process because of their simple structure. Other kinds of repetitive DNA where the repeats are not all lined up next to each other (such as those resulting from transposon activity) are more complex, and usually we do the same strategy we use with all of these other types of sequences, which is we compare against the set of known transposon sequences with potential variation.

This page titled [1.5: Gene Annotation](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta \(Consortium of Academic and Research Libraries in Illinois \(CARLI\)\)](#).

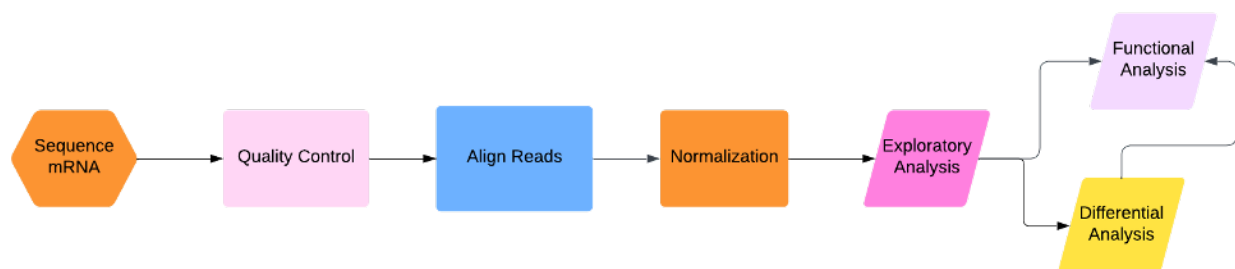
1.6: Gene Expression Analysis

What is gene expression analysis?

The mere DNA sequence is not the end of the story as we transmit biological information from DNA to ultimately make proteins. The DNA has to be "expressed" (aka, mRNA has to be made), and then that mRNA has to be read to create a protein. Gene expression analysis measures the *amount* of mRNA made for a particular gene in a particular sample. Measuring this amount tells us if the gene is turned "on" or "off" (aka whether or not the protein that the gene codes for is actually made in that sample), and also *how much* protein can potentially be made in that sample. We can use this information to, for example, differentiate between cell types, understand the functions of particular genes, and diagnose diseases related to gene expression.

Basic Strategy

The basic strategy of gene expression analysis is described in the figure below:



First, we have to actually sequence the mRNA using RNA sequencing. This process is much more complicated than DNA sequencing for two major reasons: 1) RNA is a much less stable molecule than DNA and 2) we actually want to know the amount of RNA in the sample (as opposed to DNA sequencing, when we are usually just looking for the sequence and don't care how many reads there are of each part as long as there are enough of them).

Second, quality control. Experiments are not perfect, and you don't want to work with garbage. So throw out the really bad reads. (Note: some people put this step after alignment, and some people put this step in with normalization. The reality is that you should probably be checking for quality of your data and the success of your analyses at each step.)

Third, once we have the mRNA sequenced, we have to align those sequences to the DNA sequence. This part is basically the same as what we saw in Chapter CHAPTER, though in the section discussing this part, I will go over what the current industry standard method is and why it is particularly good for RNA-seq data.

At this point, you would think you might have a pretty good idea of which parts of the DNA are being expressed, and how much (more mRNA reads aligned to one part of the DNA sequence means that that part is expressed more, right?)

No! Fourth, we have to do a bunch of normalization and adjustment for statistical and experimental variation! This is the most complicated part of the whole process.

Finally, we can do the fun stuff: analyzing the data. I will discuss three steps of this process: exploratory analysis (aka do a PCA), differential analysis (aka compare expression levels across different samples directly), and functional enrichment analysis (aka what "types" of genes are highly expressed in the sample?). I have arranged these three types of analysis at the end of the flowchart above in a strange-looking way; my opinion is that you should always do exploratory analysis first, and then from there you can go straight to functional analysis (if you are not comparing across multiple samples), straight to differential analysis (if you explicitly want to compare multiple samples), or differential analysis and then functional analysis (if you want to compare multiple samples and then use gene ontology information to characterize that comparison).

Getting your mRNA sequences

RNA is much less stable than DNA, and mRNA is single-stranded (which means that unless there's a lot of molecular machinery already floating around to handle it it can bind to itself and be very difficult to sequence). So, the first step in getting mRNA sequences is actually turning them *back* into DNA using **reverse transcriptase**. The first time you do this, you create a strand of DNA that is *complementary* to the mRNA strand, which is why this is called **cDNA** (or complementary DNA). Now you have a

double-stranded molecule, one strand of which is mRNA and one strand of which is cDNA. You then have to heat it all up a bit to detach the mRNA from the cDNA, and then reverse transcriptase will create a new strand of DNA that is complementary to the cDNA (which therefore matches the original mRNA sequence) and now you have double-stranded cDNA that contains the original mRNA sequence and its complement. What you do with it after this is outside the scope of the class, as are the details of this process in general, but you can find out more [here](#).

The old-school thing to do once you have your cDNA is to use a microarray (a grid of targets) and have your cDNA bind to specific target sequences in your microarray. The amount of cDNA that binds to your target sequences allows you to quantify how much cDNA of that sequence. Now, I'm using "quantify" a bit generously here. Microarray data comes in the form of images, where each specific target sequence has a particular spot on an array that glows more if there is more binding. The measurement of gene expression levels from microarrays is literally just measuring how bright different target sequences are. You can compare relative brightness, but there is a limit to how much quantifiable information you can actually get from a microarray experiment, and early on it was almost impossible to standardize experimental practices in order to create reproducible results (see Box BOX). Things are better now, and there are circumstances where microarray analysis is useful for gene expression analysis, but for the remainder of this chapter I will focus on the "modern" way of doing this: RNA-seq. For RNA-seq, you basically just create a cDNA library from your sample, and then sequence that cDNA using DNA sequencing techniques. The difficult part is quantifying the reads. In DNA sequencing, you normally just care about read depth to the extent that you have enough coverage to be confident in your sequencing. For RNA sequencing, the specific number of reads that map to a particular gene is actually important, and depends strongly on the experiment itself. We'll get to how we manage that, but first, let's actually align and assemble the cDNA reads.

? Exercise 1.6.1

One thing I didn't mention previously about making cDNA is: there's a ton of different kinds of RNA in a cell, and the vast majority of it is not mRNA (most of it is rRNA, and there is also a bunch of non-coding (nc) RNA). How do we isolate mRNA?

1. Consider the structure of mRNA. Is there one obvious part that is unlikely to be found in other RNA sequences?
2. rRNA is extremely highly conserved across organisms (aka the sequence doesn't change much). How can we use this to remove rRNA from the sample?
3. Which method do you have to use in prokaryotes?

Answer

1. The poly-A tail.
2. You can create small DNA probes that match rRNA sequences (because we basically know exactly what all rRNA sequences look like).
3. Prokaryotes do not put poly-A tails on their mRNA, so you have to go with the second method: removing rRNA.

Quality Control

The quality control for RNA-seq is essentially the same as it is for DNA-seq, which I discussed in Chapter 3.

Aligning and Assembling Reads

Once you have your raw RNA-seq data---the individual reads---you have to figure out what genes they come from. To explain how to do this, I'll go through the method for one of the more common tools used for this purpose: [Bowtie](#). Note that the newer method [Bowtie 2](#) is better-suited for modern sequencing methods that generate longer reads, and so if you are actually doing this I'd recommend using Bowtie 2. The algorithm is more complicated and beyond the scope of this class, so I'm using Bowtie here instead.

We have discussed alignment techniques before in Chapter 5. Bowtie is an algorithm that is specifically good at aligning very short reads to very long reference sequences (which is what the general mapping of RNA-seq reads entails) and also allows for large gaps in the middle of the mapping of individual reads (which is very useful for RNA-seq because the reads lack introns, which the reference genome still have, so you necessarily expect big gaps in the middle of your read alignments).

The first thing Bowtie does is "index" the reference genome. Instead of brute-force searching throughout the whole reference genome for each read, it transforms and stores the sequence in a special way that makes it much easier to find where a short read can potentially align. To do this, it uses an algorithm called the Burrows-Wheeler transformation.

Burrows-Wheeler Transformation

The Burrows-Wheeler (BW) transformation is a method that takes a string (a sequence of characters, or letters, or bases in this case) and rearranges all the letters in a way that makes it

1. Easier to store in memory
2. Possible to reconstruct the entire original string perfectly

This is a process that is called *lossless compression*. If you've ever seen a .zip file, that's an example of lossless compression; all the information is stored in a way that takes up less space and is unusable until you decompress it, but when you do decompress it you get all of that information back.

? Exercise 1.6.1

Consider the string AAAAAAAAAA.

1. How might you write this string (using only letters and **numbers**) in a way that keeps all the information but takes up less space?

Now consider the string ACTAGCCAA.

2. Use the method you found in Q1 to "compress" this string. How successful was that compression?

Answer

1. 9A (or A9, but this is a slightly worse way to do it because you have to backtrack to recover the original string). Congratulations! You have just discovered a compression technique called run-length encoding!
2. ACTAG2C2A (or ACTA2G2CA). Same number of characters, so this compression wasn't very good. Compression works best when there are a lot of repeated characters in a row.

The Burrows-Wheeler transform itself is so simple you can do it by hand! First, let's start with a string, say POTATO. For technical reasons, we need a unique character that isn't in the string at one or both ends of the string; typically people use "\$", and it is typically at the end of string. So let's call the string POTATO\$. The first thing to do is to take the last character of the string and put it at the front, and then repeat until you've gone all the way around:

POTATO\$
\$POTATO
O\$POTAT
T\$POTA
A\$POT
TATO\$PO
OTATO\$P

Now we sort these rearrangements in lexicographical order (aka alphabetical order but with a defined ordering of non-alphabetical symbols; in this case we only need to say that "\$" comes either before or after the alphabet. For this example I chose "before".). And then we read off the last letter of each of these sorted rearrangements (in bold) , and that's our transform.

\$POTATO
ATO\$POT
O\$POTAT
OTATO\$P
POTATO\$
TATO\$PO
T\$POTA

So our BW transform of POTATO\$ is **OTTP\$OA**. Notice how the two Ts are grouped together. If your string has a lot of repeated complex patterns, these groupings will occur frequently in this transformation, and so we can compress it better than the original string.

? Exercise 1.6.1

Find the Burrows-Wheeler transform of MISSISSIPPI\$.

Answer

IPSSM\$PISSII

OK well we mixed up our string in a weird way. Now what? Well, as it turns out, the actual string isn't that important; it's the *process* of getting that string that gives us a useful tool.

The first thing I should say is that I am going to do all the next steps by inspecting the sorted list from the BWT, but the point of this algorithm is that there are nice mathematical formulas for computing these things that we are going to do visually.

Let's take a look at this list, which is the key part of the BWT

\$POTATO
ATO\$POT
O\$POTAT
OTATO\$P
POTATO\$
TATO\$PO
TO\$POTA

Let's say I wanted to count all occurrences of the pattern "TATO" in my string. Luckily, this list is ordered alphabetically and uniquely, so every occurrence of that pattern is going to be right next to every other occurrence. We start by doing something bizarre: start with the last letter. (I'll show you why this is important in a second). So, how many occurrences are there of the pattern "O"? Then "TO"? Then "ATO"? Then "TATO"? I'll put this in a table for ease of reading:

O	TO	ATO	TATO
\$POTATO	\$POTATO	\$POTATO	\$POTATO
ATO\$POT	ATO\$POT	ATO\$POT	ATO\$POT
O\$POTAT	O\$POTAT	O\$POTAT	O\$POTAT
OTATO\$P	OTATO\$P	OTATO\$P	OTATO\$P
POTATO\$	POTATO\$	POTATO\$	POTATO\$
TATO\$PO	TATO\$PO	TATO\$PO	TATO\$PO
TO\$POTA	TO\$POTA	TO\$POTA	TO\$POTA

Luckily, in this case there is only one "TATO", so we can align that sequence perfectly. To actual get the location of "TATO" in the original string you have to do another (relatively) simple calculation to reverse the BWT.

Now why did we go through that whole rigmarole of starting with the last letter? Take a look at the number of matches. We started with 2, then we had 1, then 1, then 1. In fact, this process has the property that the number of matches *never increases* as you go from the end to the beginning! So the instant we got to "TO", we could have stopped, because "TO" is unique and has only one occurrence, and so if we know where "TO" is, we know where "TATO" is. An important thing in all of these algorithms is that you want to make them as efficient as possible, as we are dealing with very large strings and very large sequences. If you can stop before you have to go through all possible choices, then you stop.

Now, for a large sequence, this list of ordered permutations is also large, but *all of the information in this list is encoded in the BWT*. So we can take the BWT string, do some simple mathematical operations on it, and perform the same procedure I just showed you visually in this example. I keep saying "simple" here and I mean it; you basically just construct a few tables by counting and then look up numbers in those tables.

? Exercise 1.6.1

Using the ordered list of permutations, count how many occurrences there are of "ISSI" in MISSISSIPPI\$. Did you have to go through all four letters, or could you have stopped earlier? Explain why.

Answer

There are two (obviously), but in this case you did have to go through all of them because there was more than one.

So that's an efficient method for finding *exact* matches in short strings. But what about mutations or sequencing errors? Well, Bowtie allows for this by *backtracking*. Let's try to align the sequence "TOTO":

O	TO	OTO	TOTO
\$POTATO	\$POTATO	\$POTATO	\$POTATO
ATO\$POT	ATO\$POT	ATO\$POT	ATO\$POT
O\$POTAT	O\$POTAT	O\$POTAT	O\$POTAT
OTATO\$P	OTATO\$P	OTATO\$P	OTATO\$P
POTATO\$	POTATO\$	POTATO\$	POTATO\$
TATO\$PO	TATO\$PO	TATO\$PO	TATO\$PO
TO\$POTA	TO\$POTA	TO\$POTA	TO\$POTA

Oops! We can't! There is no exact match. What Bowtie does in this situation is it *backtracks*, so it stops, substitutes a random letter, and tries again. In this case, if it substituted "A" for the problem letter (that "O"), then it would find a match and align "TOTO" with the "TATO" part of "POTATO", with a slight penalty for having to make a substitution.

Notice I haven't said anything about gaps yet. That's because Bowtie doesn't directly address gaps; it assumes the reads are short enough that you don't cross a full intron with them. TopHat (built on top of Bowtie...get it?) and Bowtie 2, on the other hand, explicitly handle gaps caused by introns.

Once the reads are aligned, you have to assemble them into **transcripts**. This process is similar to what we saw in Chapter CHAPTER, but easier because you've already aligned stuff and also more difficult because you have to take into account alternative splicing. An example of a program that does this is Cufflinks.

Normalization

There are number of potential problems that occur when we do RNA-seq that we don't necessarily have to worry about for DNA-seq, mostly because we care about the actual amount of reads that map to particular parts of the genome. These problems include:

1. Longer genes will get more reads mapped to them
2. There is no real baseline control for gene expression (i.e. all genes are expressed differently everywhere)
3. Different experimental conditions can lead to more or less mRNA produced and sequenced overall

To combat these problems, there are several different normalization techniques.

Correcting for Gene Length Bias

The most common measure for correcting for gene length bias is RPKM. This stands for "reads per kilobase per million mapped reads". So, you take the number of reads that map to a particular transcript, divide that by the length of that gene in kilobases (thus normalizing for gene length) and then divide that by how many million reads were mapped to the genome in total (thus normalizing for overall sequencing depth/success).

So, if out of 4 million total mapped reads, 96 mapped to a transcript for Gene A, and that gene is 3kb long, then $RPKM = \frac{96}{4 \times 3} = 8$.

If you have paired-end reads (see Chapter CHAPTER), then you can have either two paired reads that map to the same fragment (which would be double-counted by RPKM) or one read that maps to a fragment and the other read fails to map. So if you count fragments instead of reads, that solves the potential double-counting problem and you get FPKM, or "fragments per kilobase per million mapped reads".

It will be easier for me to explain TPM (transcripts per kilobase per million mapped reads) if I start writing out equations. Call the number of reads for a particular transcript "j" r_j , and the length of gene "j" ℓ_j . Then, we compute RPKM of gene "j" as:

$$RPKM_j = 10^9 \frac{r_j}{\ell_j} \frac{1}{\sum_{i=1}^N r_i}. \quad (1.6.1)$$

So we normalize for gene length and normalize for sequencing depth in some sense "simultaneously", without considering the possibility that sequencing depth can potentially affect the relationship between the number of mapped reads and gene length. We can write TPM as follows:

$$TPM_j = 10^6 \frac{r_j}{\ell_j} \frac{1}{\sum_{i=1}^N \frac{r_i}{\ell_i}} = 10^6 \frac{RPKM_j}{\sum_{i=1}^N RPKM_i}. \quad (1.6.2)$$

Basically what TPM does is it normalizes the RPKM value of a single gene by the total RPKM value of all genes.

So, which one to use? As it turns out, none of these simple methods of normalization are very good. Why? Well, first of all, comparing these values across samples assumes that you have the same amount of total RNA, which is generally fine if you do your experiments carefully, under the same conditions, at the same time, but not guaranteed. Also, all of these methods correct for sequencing depth but no other potential variation between samples or experiments. Modern techniques like those used by the common software packages limma, DESeq2, and edgeR, rely on looking at the whole distribution of reads instead of making simplifying assumptions like RPKM and TPM do. Furthermore, there is recent debate that with the cost of RNA-seq decreasing, we don't have to rely on fancy normalization techniques if we just design our experiments properly, with replicates and large sample sizes. At any rate, people still use these values and these packages all the time, so the exercise below is about computing RPKM and TPM, and the R activity for this chapter will use DESeq2. Because different methods will give you different results, I would recommend using multiple packages on the same data if you do an actual analysis.

? Exercise 1.6.1

The following table contains data from a hypothetical experiment where RNA-seq was performed on samples from 3 different tissues for three different genes. The units are in millions of reads (this is a very fake dataset!) Fill out the RPKM and TPM values in this table and answer the following questions:

Gene	Gene Length	Heart	Brain	Lungs	RPKM Heart	RPKM Brain	RPKM Lungs	TPM Heart	TPM Brain	TPM Lungs
Gene A	10 kb	1.2	1.2	0.4						
Gene B	150 kb	0.5	3.4	2						
Gene C	75 kb	1.1	0.02	0.9						

1. Which gene is most highly expressed in each tissue (use RPKM)?
2. Which tissue is each gene most highly expressed in (use RPKM)?
3. Do your results change when you use TPM? If so, explain why. If not, explain why not.

Answer

Gene	Gene Length	Heart	Brain	Lungs	RPKM Heart	RPKM Brain	RPKM Lungs	TPM Heart	TPM Brain	TPM Lungs
Gene A	10 kb	1.2	1.2	0.4	0.043	0.026	0.012	0.87	0.84	0.61
Gene B	150 kb	0.5	3.4	2	0.0012	0.005	0.004	0.024	0.16	0.2
Gene C	75 kb	1.1	0.02	0.9	0.0052	5.8e-05	0.0036	0.11	0.002	0.18

1. Heart: A, Brain: A, Lungs: A
2. A: Heart, B: Brain, C: Heart

3. Yes! TPM switches B and C both to Lungs. This is because the total RPKM for Lungs is smaller than that for Brain, so dividing by that will increase the Lung values potentially over the Brain values.

Controlling for Batch Effects

A "batch effect" is anything that differs from one experiment to the next. We don't necessarily know what a particular batch effect is, we just know that if we do different experiments, *something* about the experiments might be different. Maybe its the incubation temperature, maybe its where you put the flask on the table, maybe its airflow turbulence patterns, maybe its the amount of coffee the scientist had before performing the experiment. This is why it's best practice to try to do all your experiments at once in as consistent an environment as possible. Batch effects are, in general, however, unavoidable, so it's good to check for them and correct for them.

The basic idea behind correcting for batch effects is a variance partition: the response variable you are measuring (gene expression in this case) varies in your data. Some of that variation is due to the stuff you are testing (e.g. treatment vs control), and some of it is due to random noise. Some of it could also be because of batch effects. In general, you can take the amount of variation in your response variable and partition it among the possible sources of variation (including the effect you are studying); this type of calculation is called a variance partition and is used across sciences. The first step in correcting for batch effects is seeing how bad they are: that is, how much of the observed variation is due to batch effects as opposed to the treatment effect?

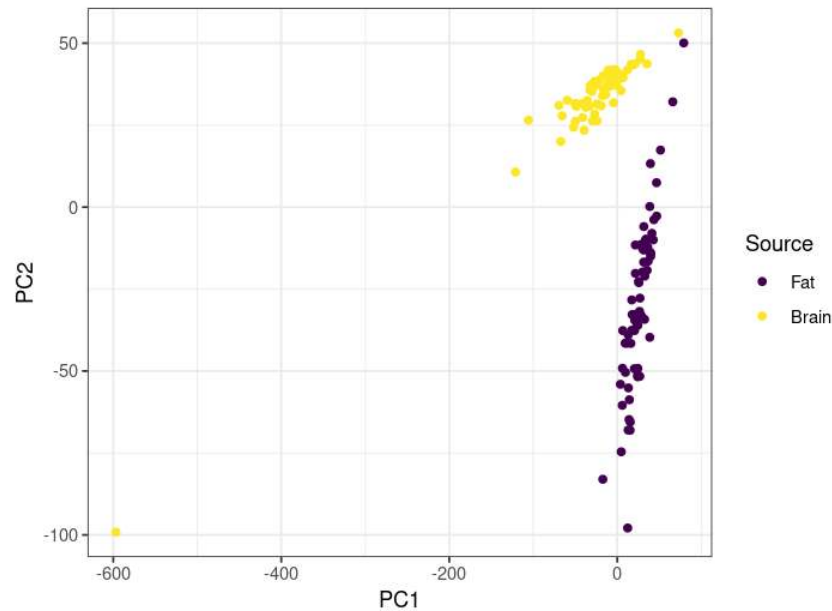
An R package that can be used to control for batch effects is BatchQC, which incorporates two of the popular control methods: COMBAT and COMBAT-seq. The following example is from an example dataset in the BatchQC package (the "Bladderbatch" dataset). This dataset consists of 57 bladder samples with 5 batches. There are 3 types of samples (cancer, biopsy, control). Batch 1 contains only cancer, 2 has cancer and controls, 3 has only controls, 4 contains only biopsy, and 5 contains cancer and biopsy. It is important to consider experimental setups like this because sometimes you have to do different treatments in different batches for logistical purposes. If so, it is crucial that you don't have perfect matching between treatments and batches, or else it would be impossible to isolate batch effects from treatment effects.

Analyzing Gene Expression Data (Finally!)

Exploratory Analysis

The first thing you should do whenever you get any data is: plot it. Visualize it somehow. See if it makes sense. Gene expression data is no different. Unfortunately, it is "high-dimensional", which means you can't just plot it on a graph with two axes. Typically, we use PCA (see Chapter CHAPTER: Additional Methods) to visualize these data.

The figures in this section are from data from [this](#) study, which looked at gene expression in two types of tissue (fat and brain) in the Eastern bumble bee *Bombus impatiens*, comparing situations where the bees were exposed to toxic heavy metals (arsenic, cadmium, chromium, and lead) to those that weren't. Here is a PCA of all samples, colored by the tissue they came from, with the observations for the PCA being the individual (normalized) gene expression values.



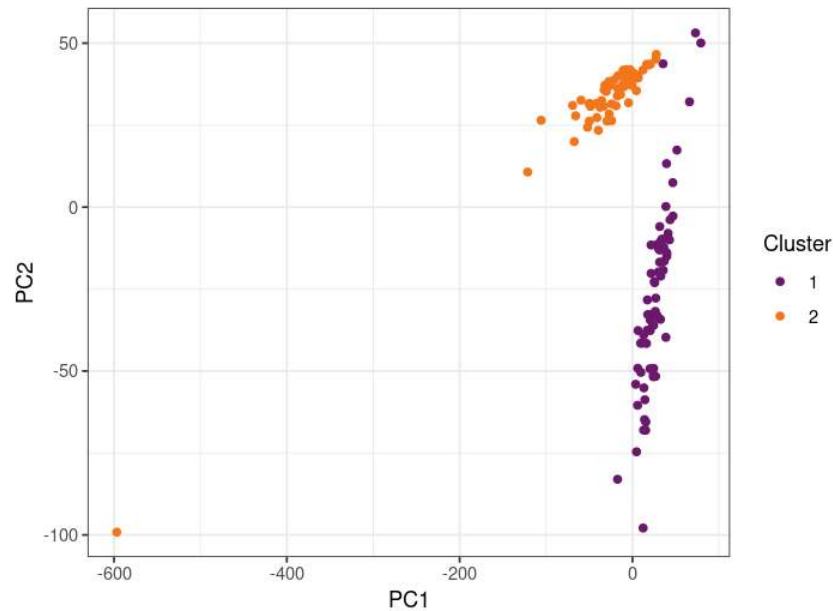
? Exercise 1.6.1

1. Does the PCA here provide a good sanity check that the samples from different tissues have different gene expression profiles? Explain why or why not.
2. Are there any samples you would want to look at more closely to see if everything worked correctly? If so, which ones? Explain your choice.

Answer

1. In general, yes. The two tissue types are well-separated by the PCA. The fact that the results appear to lie on a line for each is a bit weird, and there appears to be significant variation of gene expression profiles within tissue type, but I would say yeah this at least looks fine.
2. The one crazy brain one on the bottom left is a huge outlier, so that one is worth checking. In addition, the two closest fat and brain ones are worth checking as well.

Another thing you can do is try to cluster the data using a formal clustering method. Let's see how k-means clustering (Chapter CHAPTER: Additional Techniques) groups the samples in this dataset (here I have set $k=2$ because we know we have two different sources):



? Exercise 1.6.1

How well do the clusters match the source? Does this make you feel good or bad about the results?

Answer

Almost perfectly! In fact, here is a "contingency" table, which shows correct results as well as errors:

	Cluster 1	Cluster 2
Brain	3	57
Fat	67	0

That's a pretty good separation!

Differential Expression Analysis

Now that we have quantified our gene expression, we might want to compare expression of different genes across samples. Are some genes more or less expressed in different samples? The most basic way to do this is to look at **fold change**, which is just how much bigger one value is than another in a ratio. For example, 10 is two-fold bigger than 5 because $10/5 = 2$. Similarly, 5 is 0.25-fold bigger (or 4-fold smaller) than 20, because $5/20 = 0.25$. So if you take, say, the RPKM value (btw don't do this! use a normalized count from a software package instead!) for one gene from one sample and divide it by the RPKM value from one gene from another sample, you get the "fold-change" of expression between samples. Now, if you want to scale this to a particular multiple (usually 2), then you take the $\log_2$ of the ratio. So the 2-fold change between 2 and 10 is 1, and the 2-fold change between 20 and 5 is -2. Taking the log like this is preferable to using absolute ratios because 1) no change leads to a value of 0 and 2) you can interpret the number in units of doubling.

? Exercise 1.6.1

Fill out this table by computing the 2-fold change of expression from heart to brain tissues of each gene:

Gene	Heart	Brain	2-Fold Change
Gene A	1.2	1.2	
Gene B	0.5	3.4	

Gene C	1.1	0.02	
--------	-----	------	--

Answer

Gene	Heart	Brain	2-Fold Change
Gene A	1.2	1.2	0
Gene B	0.5	3.4	2.77
Gene C	1.1	0.02	-41

Now, there is a huge problem with stopping here: we aren't taking into account any randomness or error! We need some statistics! Let's go back to count normalization, because now we're going to be doing what the package DESeq2 does.

The DESeq2 method of normalization uses the following steps:

1. Compute the geometric mean of total raw counts across all samples for each gene. The geometric mean is the n th root of the product of n numbers. So the geometric mean of 5, 8, 2, and 4 is $(5 \times 8 \times 2 \times 4)^{\frac{1}{4}} = 4.23$. Note that if you have a count of 0 anywhere, you will get a 0 for this number. That's ok, we'll deal with that in a later step. This number is called a **pseudoreference**.
2. Compute the ratio of each sample count to this pseudoreference.
3. Take the median of all these ratios across each gene for each sample. This is where you throw out the ones with the 0s, as they would have 0 pseudoreference and therefore an infinite ratio, so we get rid of those. The result is called the **normalization factor**.
4. Divide the raw counts by the corresponding normalization factor for each sample.

Why do we go through this whole procedure? We are now actually using a statistical model instead of just making stuff up. The basic idea of the model is that

1. Most genes are *not* differentially expressed across samples.
2. The distribution of expression of genes in a sample is *overdispersed*, meaning that there are more extreme values than you would expect from a normal distribution.

The specific distribution used by DESeq2 and edgeR is the **negative binomial** distribution, as this exhibits overdispersion, but that's not the only possible choice.

Suppose a gene is expressed approximately the same across samples. Then the pseudoreference will be approximately equal to its count number in each sample. The ratio in step 2 would be approximately equal to 1, and most genes would be around this value. Thus, when you take the median in step 3, that median will *also* usually be around 1 no matter what the extreme outliers in differential expression are. The normalization factor would then be...you guessed it! Around 1, so the normalized counts would be similar to the original counts. But if there is generally lower (or higher) read depth for a particular sample (not specific to a particular gene, but the whole sample), then the normalization factor will be a little bit lower or higher than 1, respectively, and that will adjust the raw counts to account for sequencing depth.

? Exercise 1.6.1

Repeat this exercise but compute the normalized counts (NC) using the DESeq2 method and answer the following questions (ignore gene length):

Gene	Heart	Brain	Lungs	Pseudoreference	Heart/Pseudoreference	Brain/Pseudoreference	Lungs/Pseudoreference	Heart NC	Brain NC	Lungs NC
Gene A	1.2	1.2	0.4							
Gene B	0.5	3.4	2							
Gene C	1.1	0.02	0.9							

Normaliz ation factor										
-----------------------------	--	--	--	--	--	--	--	--	--	--

1. Which gene is most highly expressed in each tissue?
2. Which tissue is each gene most highly expressed in?

Answer

Gene	Heart	Brain	Lungs	Psuedore ference	Heart/Pse udorefer ence	Brain/Ps uedorefer ence	Lungs/Ps uedorefer ence	Heart NC	Brain NC	Lungs NC
Gene A	1.2	1.2	0.4	0.83	1.45	1.45	0.48	1	1	0.36
Gene B	0.5	3.4	2	1.5	0.33	2.27	1.33	0.23	2.34	1.5
Gene C	1.1	0.02	0.9	0.27	4.07	0.074	3.33	0.76	0.014	0.68
Normaliz ation Factor					1.45	1.45	1.33			

1. Heart: A, Brain: A, Lungs: B
2. A: Heart and Brain, B: Brain, C: Heart

The actual statistical model used by DESeq2 and edgeR looks basically like this:

$$K_{ij} \sim NB(s_j q_{ij}, \alpha_i) \quad (1.6.3)$$

So that looks like nonsense. Let me explain. K_{ij} is the raw count of gene "i" in sample "j". The " $\sim$ " means "is distributed as", and the NB stands for the negative binomial distribution. The negative binomial distribution has two parameters and in this case they are the mean (the first one) and a measure of how spread out the distribution is (the second one). In Chapter CHAPTER: Additional Techniques, I have an interactive app that you can use to see how changing these parameters changes the shape of the distribution. Anyways, s_j is the normalization factor from step 3, α_i measures how variable the counts are across samples for gene "i" (and is estimated from data), and q_{ij} is proportional to the "true" amount of cDNA in the same for that gene. Now, q_{ij} is *further* broken down as follows: $q_{ij} = 2^{x_j \beta_i}$, where x_j is the experimental condition (so it could be "treatment" or "control", if you're only studying one thing) and β_i is the 2-fold change between the treatment and control (if you are comparing multiple different things, the meaning of this quantity is slightly different).

So! What do you do with all that? You take your actual counts, you estimate your normalization and dispersion factors from the overall data, and then you fit this model to get those β_i values. This technique will also get you p-values for each estimated fold change. A **p-value** is the probability that you would get an equivalent or more dramatic fold change under the null model where the treatment had **no effect** (aka the parameter x_j has no effect; you can compute this by taking the distribution you get using all the fitted parameters but setting all x values to be equal, and seeing what the probability is that you get your same result under that model. This is not what they do; they do a form of t-test called a Wald test, but I'm not going to go into that here. The point is you get a p-value.).

And you get a p-value for every gene for each sample. Now comes the most boring but super important part of these types of large-scale hypothesis testing.

Multiple Testing Corrections

A p-value of 0.05 means that even if there was no effect of the treatment on gene expression at all, 5% of the time I would detect an effect.

Suppose there is in fact no effect of the treatment.

If I'm just doing one test, then 95% of the time (assuming there is no effect of the treatment) I'd see no effect, and go "ok well let's try something else". Now, if I re-did the experiment 100 times to fish for a result and got 5 false positives over those hundred experiments and chose to report only those 5 to the scientific community, that would be bad practice. In fact it would be research fraud, and if people found out I'd be (hopefully) blackballed from the scientific community.

But! If I test 1000 genes for an effect in a single experiment, I'd get 50 results! I would get super excited and go report my findings to the scientific community! Look, I found 50 genes that might do something under this treatment! And then others would try to replicate my results, get nothing, go look at my paper, see that I didn't adjust for the fact that I did many many tests at once, and then I'd (hopefully) have to retract my paper, embarrassed.

So, when you perform many different tests on the same hypothesis on the same data, you have to use a different level of scrutiny for what constitutes "statistically significant" so that you don't accidentally get excited and think you found something when you very likely did not.

The easiest, most brutal, most "conservative" (aka least likely to give you a false positive) way to do this is called the **Bonferroni correction**. This one is simple; instead of a significance level of, say, 0.05, you just divide that significance level by the total number of tests you are doing. So in the above example, if I did 1000 genes, my new significance level is $0.05/1000 = 0.00005$, and I'll call any result with a p-value below that new value "statistically significant". In this case, under the assumption that there is no effect of the treatment, I would now expect to get **1 false positive in my entire experiment only 5% of the time** (as opposed to 50 false positives every time)! What we have effectively done here is turn the false-positive rate of an individual test into the false positive rate of *the entire experiment*. This is the safest way to make this correction; when in doubt, do this.

However! In many cases, the Bonferroni correction is too strict. I mean, one false positive in my entire experiment only 5% of the time? That probably means I'm getting a *lot* of false negatives; I'm throwing away potential discoveries just to make sure I don't report any false ones. Perhaps it would be better to, instead of worrying only about false positives, worry about false *discoveries* instead.

$$\text{False positive rate} = \frac{\text{False positives}}{\text{False positives} + \text{True negatives}} \quad (1.6.4)$$

$$\text{False discovery rate} = \frac{\text{False positives}}{\text{False positives} + \text{True positives}} \quad (1.6.5)$$

Controlling for the **False Discovery Rate (FDR)** is less stringent than controlling for the false positive rate (or Type-I Error Rate, or significance level, they're all the same), but it focuses on the stuff you are planning to report out to the scientific community anyways and means that you throw away fewer potential true positives in an effort to minimize the false ones.

The most common way to control for the FDR is the **Benjamini-Hochberg (BH)** procedure, which is actually kind of fun to calculate.

Let's start with p-values 0.001, 0.002, 0.2, 0.1, 0.5, 0.25, 0.02, 0.025, 0.015, 0.8 and a significance level of 0.05.

1. Sort the P-values	0.001	0.002	0.01	0.03	0.04	0.1	0.2	0.25	0.5	0.8
2. Multiply by the number of samples*	0.01	0.02	0.1	0.3	0.4	1	2	2.5	5	8
Rank	1	2	3	4	5	6	7	8	9	10
3. Divide by rank	0.01	0.01	0.033	0.075	0.08	0.167	0.29	0.3125	0.56	0.8

*this is equivalent to the Bonferroni correction; instead of dividing the significance level and comparing the p-values to it, you are multiplying the p-values and comparing them to the old significance level

For the Bonferroni correction, we compare the results in step 2 to 0.05, and for the BH procedure we compare the results from step 3 to 0.05.

? Exercise 1.6.1

For the above example, how many tests are statistically significant

1. Unadjusted?
2. Under the Bonferroni correction?
3. Under the BH procedure?

Answer

1. 5 (0.001, 0.002, 0.01, 0.03, 0.04)
2. 2 (0.01, 0.02)
3. 3 (0.01, 0.01, 0.033)

OK, we'll finish off this section with an exercise that could be the output of a Differential Expression analysis

? Exercise 1.6.1

Perform p-value adjustments on the rest of this table and indicate which genes show a significant fold-change difference for the treatment vs the control for a significance level of **0.03**.

Gene	Control	Treatment	2-Fold Change	P-value	Bonferroni p-value	BH p-value
Gene A	10	2		0.02		
Gene B	1067	1264		0.0045		
Gene C	25	972		0.001		
Gene D	82	70		0.03		

Answer

Gene	Control	Treatment	2-Fold Change	P-value	Bonferroni p-value	BH p-value
Gene A	10	2	-2.3	0.02	0.08	0.027
Gene B	1067	1264	0.24	0.0045	0.018	0.009
Gene C	25	972	5.3	0.001	0.004	0.004
Gene D	82	70	-0.23	0.03	0.12	0.03

Functional Enrichment Analysis

Once you know what genes are expressed differently in different samples, it is useful to know what they are! The typical way to do this is to find the GO or KEGG annotations and then see if there are certain annotations that are **overrepresented** in the differentially expressed genes. This essentially amounts to a statistics problem. The most basic way to approach this is to use Fisher's Exact Test in a method called Overrepresentation Analysis (ORA). To do this, you basically have to fill out a contingency table. Let's say that out of 5000 total genes, there are 500 differentially expressed genes. Of the DE genes, 10 are related to fat

storage (for instance) and the others are not. Of the non-DE genes, 60 are related to fat storage, and the others are not. The table looks like this:

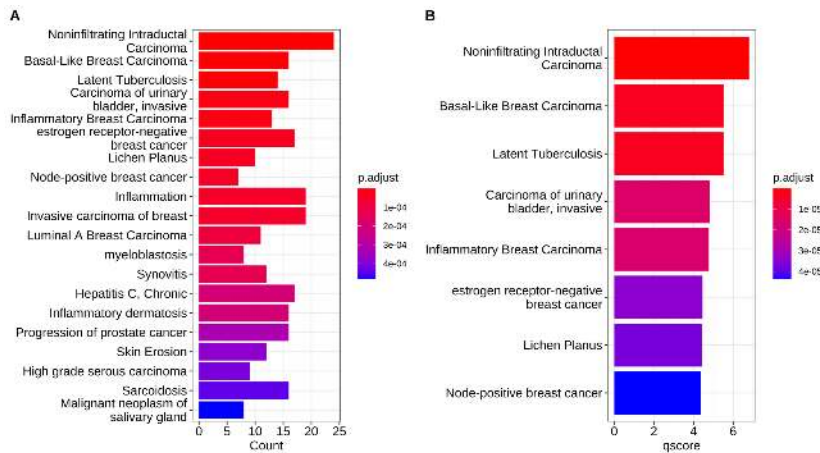
	Gene Differentially Expressed	Gene Not Differentially Expressed
Gene Related to Fat Storage	A = 10	B = 60
Gene Not Related to Fat Storage	C = 490	D = 4440

Fisher's Exact Test provides the probability that you observe at least 10 of the class of interest (DE and related to fat storage) by chance; that is, is there an increased association between fat storage and DE than you would expect by chance. It's exact because we can compute the exact probability (it's a hypergeometric distribution). In fact, here is the equation, where $N=A+B+C+D$

$$\frac{\binom{A+C}{A} \binom{B+D}{B}}{\binom{N}{A+B}} \quad (1.6.6)$$

Luckily, we don't have to compute this by hand. Remember to adjust for multiple comparisons!

There are other approaches that do not use this technique and instead focus on the rank of the gene in terms of its DE level, not necessarily the exact amount. Gene Set Enrichment Analysis (GSEA) was the first one to do this. A common one nowadays is clusterProfiler, an R package. Here is an example of a clusterProfiler result (from the online [manual](#))



Bar plots like this can show if the annotation was significantly enriched (via coloring by p-value) and how many genes correspond to that annotation. Can you guess what conditions these

Lab Activity

A lab activity called "gene_expression_lab.pdf" is found in the supplemental files for this textbook.

This page titled 1.6: Gene Expression Analysis is shared under a CC BY 4.0 license and was authored, remixed, and/or curated by Rohan Mehta (Consortium of Academic and Research Libraries in Illinois (CARLI)).

1.7: Protein Structure

So far, we have really only thought about proteins like this: CTYQVYKHPM. A sequence of amino acids. This is what we call the **primary structure** of a protein. Each amino acid has specific chemical properties that influence the overall structure of the protein. But a protein is much much more than its primary structure. First, we have to note that a protein has different structures in different environments. A protein's structure defines its function (proteins work by physically interacting with other molecules, and those interactions are dictated by structure). Almost all of these functions take place inside a cell, with a very specific cellular environment. Luckily, most of that environment is just water, so we have a pretty good understanding of the basic forces that make proteins "fold" into their structures. These chemical interactions give us **secondary** and **tertiary** structures of a protein. When we try to predict a protein's structure, we are usually looking for the tertiary structure, or if there are multiple subunits, the **quaternary** structure.

Proteins are not just long strings of amino acids. They fold over onto themselves to create complex structures. How do they do that? Well, there are chemical interactions between the amino acids that pull some together, push some apart, and connect some directly. There are chemical interactions with the external environment: for example; amino acids can be hydrophobic or hydrophilic, and because the protein is usually in water, the hydrophilic amino acids go "outside" and the hydrophobic amino acids go "inside". Other parts of the external environment, like pH, ion concentrations, and temperature, also affect the structure of the protein. Finally, many proteins require the help of *other* proteins (called "chaperone proteins") to fold properly.

In addition to all the different factors that go into the folding of a protein, there is also the problem of just how many possible structures a protein can take. Imagine taking a short string, and finding all the ways you can twist it up. Now do that with a really long string. Searching through possible protein structures is not doable. The sequence isn't enough information, and there's too much information overall. So we generally figure out protein structure by...looking at the protein.

Experimental Techniques

There are three basic experimental techniques we use to determine protein structure. This is still the standard technique today; if we have the opportunity and the resources to actually look at the protein and deduce the structure, that's the best way. We'll talk about computational techniques to *predict* protein structure in later sections, but this is still how we *validate* those predictions.

1. **X-ray Crystallography:** This is the classic technique. You make a crystal out of your protein (the same way you would make a crystal out of anything else; precipitate it from a supersaturated solution) and then fire X-rays at it. The specific arrangement of the electrons in the protein crystal diffract the X-rays, and you can deduce electron densities and subsequently the structure of the protein from the diffraction patterns. X-ray crystallography is one of those fields that is legitimately extremely difficult to get really good at, but it's the gold standard for protein structure. The main drawback is that some proteins do not crystallize well, and so this technique wouldn't work very well.
2. **(Cryo-)Electron microscopy:** This one is the easiest one to conceptualize: just look at it under a microscope! Now, you can imagine that it is very difficult to look at very tiny things, and you would be correct. You have to freeze the protein and put it on a special grid. This means that you can only get limited results for proteins with flexible structures (aka clusters of amino acids that are "floppy" and don't form rigid structures; this is actually extremely common). Also, the proteins have to be large, and the resolution tends to be low.
3. **NMR Spectroscopy:** Another common structural chemistry technique is NMR spectroscopy. What you do here is put the protein in a solution in a magnetic field and blast it with radio waves. Vibration patterns indicate which nuclei are close to each other (aka where there are bonds). From this information, you can solve for the whole structure. This method is good for proteins that do not crystallize well and for flexible proteins, but they have to be relatively small.

So what information do we actually get out of these methods? The main stuff is literally: the position of each atom in the protein, the bond angles, and any other additional information we get from these methods (like, for instance, how stiff or floppy a particular part of the protein is). This information is given in the order of the atoms in the amino acids in the protein sequence and stored in a database call the Protein Data Bank.

Protein Data Bank

The Protein Data Bank ([PDB](https://www.rcsb.org/)) was established in 1971 as an open-access database of protein structural information. Each entry not only contains "solved" protein structures, but also a lot of other information about the protein itself.

The main file format used in the PDB is a PDB file. Here is an example:

The main parts of a PDB file are as follows:

1. First, a large header that includes a lot of metadata (aka what kind of protein this is, details of the experiment, who found the structure, etc.) as well as some structural information about the protein that isn't easy to put in the next part
2. The amino acid sequence of all of the peptide chains
3. A list of all the atoms in the protein, which amino acid they are in, their 3D positions, and some chemical information about them.
4. A list of other atoms that are not part of the protein itself (like a ligand or an ion that affects the structure) with their positions and chemical information.

If you open up a PDB file, you may notice that it is actually pretty easy to read. That's good for us! But it's bad for computers. We can write programs that can read PDB files and display the protein structures, but PDB files have mostly been supplanted by a new format, mmCIF, which is more flexible and easier for a computer to read. New protein structures may not have PDB files associated, just mmCIF files. Two activities for this chapter explore the PDB and the PDB and mmCIF file formats.

Computational Prediction of Protein Structure

Just like with sequence assembly, there are two ways you can predict a protein structure: one way is to do it completely from scratch (ab-initio) and the other way is to use existing information from similar proteins. We'll start with the second way.

Homology Modeling

Recall that "homology" means "shared through common ancestry". There are two principles of homology modeling:

1. Protein structure is entirely determined by protein sequence (we know that this is technically not true, but the problem that all of these prediction methods are trying to solve is "what is the structure of the protein given the sequence?")
2. Protein *structure* evolves more slowly than protein *sequence* (there are silent mutations, and missense mutations might lead to similar amino acids that don't disrupt the overall structure much). Thus, sequence similarity implies strong structural similarity.

Basically, as long as the two aligned sequences are long enough and they share enough sequence similarity, you can use homology modeling to predict one structure from another. The first step is to take your protein sequence and BLAST it to find similar sequences. You then have to align your sequence with these best matches, and often a MSA algorithm is useful here if the sequence similarity is low (so you can use information from multiple different hits). Each protein contains a "backbone" of four atoms: N, C_{alpha} (the alpha-carbon), C, and O. For amino acids that are aligned to the similar sequence (called the "template"), you can simply copy over the coordinates of the matched backbones. If the amino acids themselves match, you can also copy the side chain coordinates. If they don't match, you have to try to find the best conformation of the side chain in 3D space, and you use pre-existing information about general trends in how these side chains are positioned in the context of backbones (i.e. some backbone conformations force the side chains into specific places).

To deal with potential gaps in the alignment, you have to either insert or delete some amino acids in the model compared to the template. This requires changing the backbone. One useful rule is that these backbone changes never occur inside specific **secondary structure** elements, like alpha helices and beta sheets, so you can move all of those possible changes to the outside of those structures (to areas called "loops"). Now it becomes difficult. You can either search through known protein structures to guess how these insertions/deletions will affect the structure, or you can use a model that just uses physics to minimize the energy of the structure (this is also what is used in ab-initio prediction).

To optimize the model, you can iteratively predict the backbone, then the loops/side chains, then the backbone, then the loops/side chains, etc. You can also put the physics into the model and then run a molecular dynamics situation to see where the protein "ends up" from your initial guess and the laws of physics. All of this stuff used to be done manually, with specialized 3D visualization software. Now, however, we can automate it. One major automated homology-modeling method is [SWISS-MODEL](https://swissmodel.exp.unihamburg.de/), which is available online.

Fold Recognition/Threading

This method is based on the "fold recognition problem", which states "which of the known ways a protein can fold would we predict given a particular protein sequence"? This method more heavily involves physics, and is used when there is no good homology match in the PDB. It is also called "threading" because the basic idea is we take our backbone that we get from a homology search, then we try to "thread" a different protein sequence through it. If it has a similar structure to the homologous

sequence, it'll fit through well. If it doesn't it, won't. Sometimes, however, an unrelated sequence fits very well! This is the benefit of fold recognition as an addition to/over homology; you can predict structures for some unrelated sequences.

Fold recognition is based on two principles:

1. There aren't that many types of protein folds that exist in nature (~1000).
2. The vast majority of newly-solved structures do not have newly-solved folds.

Thus, even if your protein is not closely related to other proteins, there are only a few ways in which protein folds actually happen, so you might be able to predict the structure of the protein anyways.

Ab-Initio Prediction

If you want to predict a protein *from scratch*, then the general strategy is: use our knowledge of physical and chemical forces to find a structure that has the amino acids in your sequence which minimizes the free energy of your protein (aka is in the most "relaxed" configuration). The biggest difficulty here is: there are so many possible configurations a protein can take! How do you search that space for the one with the lowest free energy?

One thing to do is simplify the search space. You can restrict how the conformation of side chains to those already found in the PDB (similar to the fold recognition strategy). You can break the protein sequence down into smaller pieces and then put them together again later. You can pretend that every atom can only occupy points on a 3D lattice. All of these strategies can make it easier to find a conformation that has low free energy, but because they are all simplifications, there's a limit to their ultimate accuracy. Sometimes, to make a problem doable, you have to simplify it enough

Once you have a search space you have to calculate the free energy of particular conformations. The function used to calculate this is called a "potential" function, and incorporates interactions between the peptides and water (or other solvent), electrostatic interactions, Van der Waals interactions, and covalent interactions (aka chemical and physical forces).

Finally, once you have your search space and the function you want to minimize, you can use any standard optimization algorithm to find the lowest free energy structure (or something close).

AlphaFold

Suppose you wanted to create a method to distinguish between dogs and cats. What information would you give it? You might give it some physical characteristics, like leg-to-head length ratio, or number of teeth, or something like that. Number of legs would be useless, because both dogs and cats have four legs in general. If you have DNA sequences, that would be even better! Once you make these observations, you can *train* a model to distinguish between dogs and cats using these observations. How do you do this? A model will have some "parameters", some numbers that change depending on how you train it. So let's say that "number of teeth < X --> this is a cat" is some part of the model. The "X" would be the parameter. If you trained this model on a bunch of dogs and cats (by figuring out which X value leads to the most accurate classification) you might come up with X = 35 (dogs in general have 42 teeth and cats have 30, with some variation like everything else). Then, you can check to see if your model is good by *testing* it on a new dataset of "unclassified" dogs and cats. One reason you want to do this is let's say you got your dog and cat teeth data from a veterinarian, which would potentially oversample from dogs and cats with fewer than normal teeth, which might make your initial best value of X to be closer to 30. If you test this on a less biased dataset, it might not do as well. So you test your model, and if it's good, great! If it's not, you either 1) find better data to train your model on or 2) change the model structure a bit (maybe add in another observation like leg-to-head ratio).

Anyways, I just described the process of machine learning. That's it. All that "deep learning", "AI" stuff? It's just that. It's much much fancier! But it's fundamentally that. A neural net/deep learning algorithm uses a very large number of very simple operations like "Is this data input + 2*this other data input > 4?" (the "parameters" in this model include which data inputs to use, that "2", and that "4"), and all the results of those operations are connected in a potentially complicated way (similar to how neurons are connected; hence the name "neural network"). Note that being very clever about the "fancy" stuff won the 2024 Nobel Prize in Physics!

So, why did I bother going through that here? Well, in the last 7 years, there has been a huge, game-changing development in protein structure prediction using these deep learning models. Every two years, there is an international protein structure prediction competition called [CASP](#). In 2018, a submission from Google's AI wing called "AlphaFold" placed first in the "free" modeling problems (where there is no homology in the PDB) and second in the "template-based" modeling problems (which use homology).

The original AlphaFold algorithm used a type of neural network called a [convolutional neural network](#) that was trained on PDB data. What it was trained to do is: predict the physical distance between the second carbons (aka C, not Ca) on the peptide backbone. (If the protein was just a big long lin, these distances would all be the same and decrease linearly with the number of base pairs between them, but if the protein is folded, then carbons that are not nearby in the sequence will be nearby in space). The hard part was figuring out what network structure worked well for this. The neural network takes in protein sequence and multiple sequence alignment (MSA) data, and puts out a distance matrix for all the second carbons. An initial guess of the overall protein structure (aka carbon positions) is deduced from these carbon distances, and then a physical force-based model is used to optimize the structure to get to the lowest energy state. AlphaFold demonstrated that you could use machine learning to predict protein structures effectively.

Then, two years later, at the next CASP, a completely different method by the same people called AlphaFold2 came around and completely destroyed the competition. This algorithm had a ton of new machine-learning innovations as well as better ways to integrate biological data.

Post-AlphaFold

In 2021, AlphaFold 2 was published in open-source form (meaning anyone can incorporate it into their methods, and many people have done so since). AlphaFold 3, released in 2024, focuses on predicting complexes: structures of multiple proteins and/or proteins and ligands or nucleic acids. In 2024, John Jumper, the head of the team that created AlphaFold 2 as well as Demis Hassabis, the head of Google's AI unit won the Nobel Prize in Chemistry, along with another guy I haven't mentioned yet: David Baker. While the DeepMind team sort of just "jumped in" (get it?) to protein folding from an AI background, Baker and his group have been working on protein folding for decades. His group's main algorithm is called Rosetta and was one of the leading ab-initio protein structure prediction methods. His group mostly works on protein design these days: creating new proteins that can be used in clinical and other settings.

Lab Activity

A lab activity called "protein_structure_lab.zip" can be found at [this link](#). It contains an RMarkdown file for the lab as well as data files.

This page titled [1.7: Protein Structure](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [Rohan Mehta](#) ([Consortium of Academic and Research Libraries in Illinois \(CARLI\)](#)) .

Index

A

accessible education

[InfoPage](#)

algorithms

[1.4: Genome Alignment](#)

alignment

[1.1: BLAST \(AKA so you have a sequence\)](#)

amino acid sequences

[Front Matter](#)

analysis methods

[Front Matter](#)

assembly evaluation

[1.3: Genome Assembly](#)

B

bioinformatics

[Front Matter](#)

[1.2: DNA Sequencing](#)

[1.3: Genome Assembly](#)

biological information

[Front Matter](#)

biological systems

[Front Matter](#)

blast

[1.1: BLAST \(AKA so you have a sequence\)](#)

BLASTn

[1.1: BLAST \(AKA so you have a sequence\)](#)

BLASTp

[1.1: BLAST \(AKA so you have a sequence\)](#)

BLASTx

[1.1: BLAST \(AKA so you have a sequence\)](#)

BUSCO

[1.3: Genome Assembly](#)

C

collaborative learning

[InfoPage](#)

college textbook affordability

[InfoPage](#)

computational biology

[1.3: Genome Assembly](#)

curriculum development

[InfoPage](#)

customizable learning materials

[InfoPage](#)

D

data analysis

[1.2: DNA Sequencing](#)

differential analysis

[1.6: Gene Expression Analysis](#)

differential expression

[1.6: Gene Expression Analysis](#)

distance matrix

[1.4: Genome Alignment](#)

DNA sequences

[Front Matter](#)

DNA sequencing

[1.2: DNA Sequencing](#)

[1.3: Genome Assembly](#)

E

educational technology

[InfoPage](#)

exploratory analysis

[1.6: Gene Expression Analysis](#)

F

functional annotation

[1.5: Gene Annotation](#)

functional enrichment analysis

[1.6: Gene Expression Analysis](#)

G

gene annotation

[1.5: Gene Annotation](#)

gene expression analysis

[1.6: Gene Expression Analysis](#)

gene ontology

[1.6: Gene Expression Analysis](#)

genetic codes

[1.5: Gene Annotation](#)

genome annotation

[1.5: Gene Annotation](#)

genome assembly

[1.3: Genome Assembly](#)

genomic data

[1.3: Genome Assembly](#)

global alignment

[1.4: Genome Alignment](#)

guide tree

[1.4: Genome Alignment](#)

H

homology modeling

[1.7: Protein Structure](#)

I

Illumina sequencing

[1.2: DNA Sequencing](#)

interactive textbooks

[InfoPage](#)

L

LibreTexts

[InfoPage](#)

local alignment

[1.4: Genome Alignment](#)

M

molecular biology

[1.3: Genome Assembly](#)

mRNA

[1.6: Gene Expression Analysis](#)

multiple sequence alignment

[1.4: Genome Alignment](#)

N

nanopore sequencing

[1.2: DNA Sequencing](#)

normalization

[1.6: Gene Expression Analysis](#)

nucleotide

[1.1: BLAST \(AKA so you have a sequence\)](#)

O

OER

[InfoPage](#)

open reading frames

[1.5: Gene Annotation](#)

P

pairwise alignment

[1.4: Genome Alignment](#)

phylogenetic trees

[1.4: Genome Alignment](#)

primary structure

[1.7: Protein Structure](#)

protein

[1.1: BLAST \(AKA so you have a sequence\)](#)

protein analysis

[Front Matter](#)

protein folding

[1.7: Protein Structure](#)

protein interactions

[1.7: Protein Structure](#)

protein structure

[1.7: Protein Structure](#)

Q

quality assessment

[1.3: Genome Assembly](#)

quality control

[1.2: DNA Sequencing](#)

quaternary structure

[1.7: Protein Structure](#)

R

regulatory sequences

[1.5: Gene Annotation](#)

RNA sequences

[Front Matter](#)

RNA sequencing

[1.6: Gene Expression Analysis](#)

S

Sanger sequencing

[1.2: DNA Sequencing](#)

secondary structure

[1.7: Protein Structure](#)

sequence

[1.1: BLAST \(AKA so you have a sequence\)](#)

sequence data

[Front Matter](#)

sequencing data

[1.2: DNA Sequencing](#)

sequencing methods

[1.2: DNA Sequencing](#)

side chains

[1.7: Protein Structure](#)

statistical adjustment

[1.6: Gene Expression Analysis](#)

structural annotation

[1.5: Gene Annotation](#)

structural biology

[Front Matter](#)

[1.7: Protein Structure](#)

T

tBLASTn

[1.1: BLAST \(AKA so you have a sequence\)](#)

tBLASTx

[1.1: BLAST \(AKA so you have a sequence\)](#)

tertiary structure

[1.7: Protein Structure](#)

transcription

[1.5: Gene Annotation](#)

translation initiation

[1.5: Gene Annotation](#)

Detailed Licensing

Overview

Title: [Introduction to Bioinformatics \(Mehta\)](#)

Webpages: 19

All licenses found:

- [CC BY 4.0](#): 94.7% (18 pages)
- [Undeclared](#): 5.3% (1 page)

By Page

- [Introduction to Bioinformatics \(Mehta\)](#) - [CC BY 4.0](#)
 - [Front Matter](#) - [CC BY 4.0](#)
 - [TitlePage](#) - [CC BY 4.0](#)
 - [InfoPage](#) - [CC BY 4.0](#)
 - [Table of Contents](#) - [Undeclared](#)
 - [Licensing](#) - [CC BY 4.0](#)
 - [1: Modules](#) - [CC BY 4.0](#)
 - [1.1: BLAST \(AKA so you have a sequence\)](#) - [CC BY 4.0](#)
 - [1.2: DNA Sequencing](#) - [CC BY 4.0](#)
 - [1.3: Genome Assembly](#) - [CC BY 4.0](#)
 - [1.4: Genome Alignment](#) - [CC BY 4.0](#)
 - [1.5: Gene Annotation](#) - [CC BY 4.0](#)
 - [1.6: Gene Expression Analysis](#) - [CC BY 4.0](#)
 - [1.7: Protein Structure](#) - [CC BY 4.0](#)
 - [Back Matter](#) - [CC BY 4.0](#)
 - [Index](#) - [CC BY 4.0](#)
 - [Glossary](#) - [CC BY 4.0](#)
 - [Detailed Licensing](#) - [CC BY 4.0](#)
 - [Detailed Licensing](#) - [CC BY 4.0](#)

Detailed Licensing

Overview

Title: [Introduction to Bioinformatics \(Mehta\)](#)

Webpages: 19

All licenses found:

- [CC BY 4.0](#): 94.7% (18 pages)
- [Undeclared](#): 5.3% (1 page)

By Page

- [Introduction to Bioinformatics \(Mehta\)](#) - [CC BY 4.0](#)
 - [Front Matter](#) - [CC BY 4.0](#)
 - [TitlePage](#) - [CC BY 4.0](#)
 - [InfoPage](#) - [CC BY 4.0](#)
 - [Table of Contents](#) - [Undeclared](#)
 - [Licensing](#) - [CC BY 4.0](#)
 - [1: Modules](#) - [CC BY 4.0](#)
 - [1.1: BLAST \(AKA so you have a sequence\)](#) - [CC BY 4.0](#)
 - [1.2: DNA Sequencing](#) - [CC BY 4.0](#)
 - [1.3: Genome Assembly](#) - [CC BY 4.0](#)
 - [1.4: Genome Alignment](#) - [CC BY 4.0](#)
 - [1.5: Gene Annotation](#) - [CC BY 4.0](#)
 - [1.6: Gene Expression Analysis](#) - [CC BY 4.0](#)
 - [1.7: Protein Structure](#) - [CC BY 4.0](#)
 - [Back Matter](#) - [CC BY 4.0](#)
 - [Index](#) - [CC BY 4.0](#)
 - [Glossary](#) - [CC BY 4.0](#)
 - [Detailed Licensing](#) - [CC BY 4.0](#)
 - [Detailed Licensing](#) - [CC BY 4.0](#)